

NETWORK INTRUSION DETECTION SYSTEM



A Project report submitted in partial fulfillment of requirements for the award of degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND TECHNOLOGY

By

**PATAN SAJID ALI KHAN (229X1A2851)
QUAZI MOHAMMED MOSA (239X5A2870)
APPAPOGU CHARITHA SRI (229X1A2836)**

Under the esteemed guidance of

Sri. P. Praveen Yadav

Assistant Professor

Department of C.S.E.

Department of Computer Science and Engineering

G. PULLA REDDY ENGINEERING COLLEGE (Autonomous): KURNOOL

(Affiliated to JNTUA, ANANTAPURAMU)

2025 – 2026

Department of Computer Science and Engineering

G. PULLA REDDY ENGINEERING COLLEGE (Autonomous): KURNOOL

(Affiliated to JNTUA, ANANTAPURAMU)



CERTIFICATE

This is to certify that the Project Work entitled “**NETWORK INTRUSION
DETECTION SYSTEM**” is a bonafide record of work carried out by

PATAN SAJID ALI KHAN (229X1A2851)

QUAZI MOHAMMED MOSA (239X5A2870)

APPAPOGU CHARITHA SRI (229X1A2836)

Under my guidance and supervision in partial fulfillment of the requirements
for the award of degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND TECHNOLOGY

.....
Sri. P. Praveen Yadav

Assistant Professor,
Department of C.S.E.,
G. Pulla Reddy Engineering College,
Kurnool.

.....
Dr. N. Kasiviswanath

Professor & Head of the Department,
Department of C.S.E.,
G. Pulla Reddy Engineering College,
Kurnool.

DECLARATION

I hereby declare that the project titled “**NETWORK INTRUSION DETECTION SYSTEM**” is an authentic work carried out by me as the student of **G. PULLA REDDY ENGINEERING COLLEGE(Autonomous) Kurnool**, during 2025-26 and has not been submitted elsewhere for the award of any degree or diploma in part or in full to any institute.

Patan Sajid Alli Khan
(229X1A2851)

Quazi Mohammed Mosa
(239X5a2870)

Appapogu Charitha Sri
(229X1A2836)

ACKNOWLEDGEMENT

We wish to express our deep sense of gratitude to our project guide **Sri. P. Praveen Yadav, Assistant Professor** of Computer Science and Engineering Department, G. Pulla Reddy Engineering College, for his immaculate guidance, constant encouragement and cooperation which have made possible to bring out this project work.

We are grateful to our project incharge **Dr. G. Rajeswarappa, Associate Professor** of Computer Science and Engineering Department, G. Pulla Reddy Engineering College, for helping us and giving us the required information needed for our project work.

We are thankful to our Head of the Department **Dr.N.Kasiviswanath**, for his whole hearted support and encouragement during the project sessions.

We are grateful to our respected Principal **Dr.B.Sreenivasa Reddy**, for providing requisite facilities and helping us in providing such a good environment.

We wish to convey our acknowledgements to all the staff members of the Computer Science and Engineering Department for giving the required information needed for our project work.

Finally we wish to thank all our friends and well wishers who have helped us directly or indirectly during the course of this project work.

ABSTRACT

In the contemporary digital landscape, cybersecurity threats continue to evolve at an unprecedented pace, with malicious actors employing increasingly sophisticated techniques to infiltrate and compromise network infrastructures. Traditional signature-based Intrusion Detection Systems (IDS) struggle to identify novel and zero-day attacks, necessitating the development of intelligent systems capable of learning and adapting to emerging threats. This project, titled "NETWORK INTRUSION DETECTION SYSTEM (NIDS)," aims to design and implement a machine learning-based framework for real-time detection and classification of network intrusions with high accuracy and minimal false positives.

The proposed system leverages live packet capture using Scapy for real-time network monitoring, combined with machine learning classification algorithms to analyze network traffic patterns and identify anomalous behavior. The architecture employs a multi-threaded pipeline design that captures packets, aggregates flows, extracts 51-dimensional feature vectors, and classifies network activities using a Random Forest classifier trained on the CICIDS2017 dataset. To facilitate practical deployment and accessibility, the system provides a REST API backend built with Flask, enabling seamless integration with external monitoring tools and security frameworks. Additionally, an interactive web-based dashboard provides real-time visualization of network statistics, active flows, and predicted security events, offering security administrators an intuitive interface for monitoring and decision-making.

The implementation encompasses three core components: the IDS core pipeline for packet processing and ML classification, a RESTful API with 12 endpoints for system control and data retrieval, and a web dashboard with AJAX-based real-time updates. The CLI tool further enhances usability by providing cross-platform command-line management capabilities. Initial evaluations demonstrate the system's ability to differentiate between benign and malicious network traffic with high precision and recall, establishing a robust foundation for practical deployment in security operations centers and enterprise networks. This work contributes to the broader effort of developing accessible, efficient, and accurate intelligent systems for network security in the face of evolving cyber threats.

TABLE OF CONTENTS

1. INTRODUCTION.....	2
1.1 INTRODUCTION.....	2
1.2 MOTIVATION.....	3
1.3 PROBLEM DEFINITION.....	4
1.4 OBJECTIVE OF THE PROJECT.....	5
1.5 LIMITATIONS OF THE PROJECT.....	6
1.6 ORGANIZATION OF THE PROJECT.....	9
2. SYSTEM SPECIFICATIONS.....	12
2.1 SOFTWARE SPECIFICATIONS.....	12
2.1.1 Functional Requirements.....	12
2.1.2 Non-functional requirements.....	12
2.1.3 Programming Languages and Core Libraries.....	12
2.1.4 Backend and API Framework.....	12
2.1.5 Frontend and Dashboard Technologies.....	12
2.1.6 Development and Deployment Stack.....	13
2.2 HARDWARE SPECIFICATIONS.....	13
2.2.1 Minimum Requirements.....	13
2.2.2 Recommended Specifications.....	13
2.2.3 Operating System Compatability.....	13
3. LITERATURE SURVEY.....	15
3.1 INTRODUCTION.....	15
3.2 EXISTING SYSTEMS.....	15
3.2.1 Signature-Based Intrusion Detection Systems.....	15
3.2.2 Machine Learning-Based Approaches.....	15
3.2.3 Deep Learning and Neural Network Approaches.....	16
3.2.4 Hybrid and Context-Aware Approaches.....	16
3.2.5 Dataset and Benchmark Considerations.....	17

3.3 DISADVANTAGES OF EXISTING SYSTEMS.....	17
3.3.1 Limitations of Signature-Based Systems.....	17
3.3.2 Computational and Deployment Complexity of Deep Learning.....	18
3.3.3 Data and Generalization Challenges.....	18
3.3.4 Integration and Operational Limitations.....	18
3.3.5 Scalability and Practical Deployment.....	19
3.3.6 Absence of Contextual Intelligence.....	19
3.4 PROPOSED SYSTEM.....	19
3.4.1 System Overview and Architecture.....	19
3.4.2 Real-Time Packet Capture and Feature Extraction.....	20
3.4.3 Machine Learning Classification.....	20
3.4.4 Operational Interface and Accessibility.....	20
3.4.5 Cross-platform Management and Deployment.....	21
3.4.6 Addressing Existing System Limitations.....	21
4. DESIGN AND IMPLEMENTATION.....	23
4.1 INTRODUCTION.....	23
4.2 SYSTEM ARCHITECTURE.....	24
4.2.1 Overview.....	24
4.2.2 Data Acquisition and Processing Pipeline.....	24
4.2.3 Machine Learning and Classification Layer.....	25
4.2.4 Operational Interface Layer.....	25
4.2.5 Data Flow Interaction.....	25
4.2.6 Threading Model.....	27
4.2.7 Configuration and State Management.....	27
4.3 Component Design.....	27
4.3.1 Packet Capture Module.....	27
4.3.2 Flow Aggregation and Timeout.....	28
4.3.3 Feature Extraction.....	29

4.3.4 ML Classification Module.....	30
4.3.5 REST API Design.....	30
4.3.6 Web Dashboard.....	32
4.3.7 CLI Tool.....	33
4.3.8 Storage and Configuration Management.....	33
4.4 IMPLEMENTATION DETAILS.....	34
4.4.1 Project Structure and Organization.....	34
4.4.2 Key Modules and Implementation.....	35
4.4.3 Model Training and Loading.....	36
4.4.4 Configuration Management.....	41
4.4.5 Integration Between Modules.....	41
4.4.6 Threading and Synchronization.....	41
4.4.7 Error Handling and Resilience.....	42
4.5 DESIGN PATTERNS AND DECISIONS.....	42
4.5.1 Random Forest over Deep Learning.....	42
4.5.2 Threading-Based Concurrency over Async/Await.....	43
4.5.3 Stateless REST API Design.....	44
4.5.4 Feature Engineering Approach.....	44
4.5.5 Database Persistence.....	45
4.5.6 Error Handling Philosophy.....	45
5. RESULTS AND TESTING.....	47
5.1 TESTING METHODOLOGY.....	47
5.2 VM DEPLOYMENT AND ATTACK SCENARIOS.....	47
5.3 CLI TESTING RESULTS.....	48
5.3.1 Info Command.....	48
5.3.2 Config Command.....	49
5.3.3 Start and status command.....	49
5.3.4 Stop Command.....	50

5.3.5 CLI Test Summary.....	50
5.4 DASHBOARD TESTING AND HISTORY INTERFACE.....	51
5.4.1 Real-time Prediction Display.....	51
5.4.2 History Interface.....	51
5.4.3 Dashboard Testing Summary.....	52
5.5 MODEL PERFORMANCE RESULTS.....	52
5.5.1 Overall Accuracy And Per-Class Metrics.....	52
5.5.2 Confusion Matrix Analysis.....	53
5.5.3 Cross-Validation Robustness.....	54
5.5.4 Feature Importance and Overfitting Elimination.....	54
5.5.5 Confidence Score Distribution.....	55
5.5.6 Model Performance Summary.....	56
6. CONCLUSION.....	58
6.1 SUMMARY OF ACHIEVEMENTS.....	58
6.2 KEY FINDINGS.....	58
6.3 FUTURE ENHANCEMENTS.....	58
REFERENCES.....	60

TABLE OF FIGURES

Figure 4.2.1.1: Architecture Diagram.....	24
Figure 4.2.5.1: Data Acquisition and Processing Pipeline.....	26
Figure 4.3.1.1: Packet Capture Flow.....	28
Figure 4.3.6.1: Web Dashboard for Admin.....	32
Figure 4.4.1.1: Project File Structure.....	34
Figure 4.4.3.1: Confusion Matrix - Test Set.....	36
Figure 4.4.3.2: Top 20 Features by Importance.....	37
Figure 4.4.3.3: Model Calibration Curve - Isotonic Calibration.....	38
Figure 4.4.3.4: Original vs. After Isotonic Calibration - Calibration Curve Comparison.....	39
Figure 5.3.1.1: IDS CLI - Info Command.....	48
Figure 5.3.2.1: IDS CLI - Config Command.....	49
Figure 1: IDS CLI - Start and Status Command.....	50
Figure 5.4.1.1: NIDS Web Dashboard.....	51
Figure 5.4.2.1: NIDS History Interface.....	52
Figure 5.5.1.1: Classification Report Table.....	53
Figure 5.5.3.1: Cross Validation Report.....	54
Figure 5.5.4.1: Top 10 Important Features.....	54
Figure 5.5.5.1: Confidence Score Statistics.....	55
Figure 5.5.5.2: Confidence Score Distributions.....	55

INDEX OF TABLES

Table 4.3.5.1: IDS REST APIs.....	31
-----------------------------------	----

INTRODUCTION

1. INTRODUCTION

The Network Intrusion Detection System (NIDS) is a machine learning-based cybersecurity application designed to detect and classify network intrusions in real-time. The system captures live network traffic using Scapy, extracts meaningful features from packet data, and employs a Random Forest classifier trained on the CICIDS2017 dataset to classify network activities as benign or malicious. By integrating a multi-threaded pipeline architecture, a comprehensive REST API, and an interactive web dashboard, NIDS provides security administrators with an accessible, scalable, and accurate platform for network threat detection and monitoring. The system enables organizations to identify anomalous network behavior, detect novel attacks, and respond to security incidents with minimal latency and maximum visibility into network operations.

1.1 INTRODUCTION

In an era marked by escalating cyber threats and sophisticated attack methodologies, network security has emerged as a critical concern for organizations worldwide. The proliferation of connected devices, cloud infrastructure, and digital transformation initiatives has expanded the attack surface exponentially, creating unprecedented challenges for cybersecurity professionals. While preventive measures such as firewalls and access control mechanisms serve as first-line defenses, they often prove insufficient against advanced persistent threats (APTs), zero-day exploits, and polymorphic malware that evade traditional signature-based detection systems.

The detection of network intrusions has become increasingly challenging due to the volume, velocity, and diversity of network traffic. Traditional Intrusion Detection Systems (IDS) rely on predefined attack signatures and manually crafted rules, making them reactive rather than proactive. These signature-based approaches struggle to identify previously unseen attack patterns, novel variants, or sophisticated multi-stage attacks that blend legitimate traffic with malicious payloads. Consequently, security operations centers (SOCs) face the dual challenge of processing massive volumes of alerts while simultaneously missing sophisticated attacks that deviate from known patterns.

The advent of machine learning and artificial intelligence presents a paradigm shift in network security. By analyzing historical network traffic patterns and learning to distinguish between benign and malicious behavior, ML-based IDS systems can identify anomalies and novel attacks with greater accuracy and adaptability than rule-based approaches. Such intelligent systems

have the potential to detect low-and-slow attacks, insider threats, and zero-day exploits by recognizing statistical deviations from baseline network behavior.

This project, titled "NETWORK INTRUSION DETECTION SYSTEM (NIDS)," represents a comprehensive effort to design, implement, and deploy a production-grade machine learning-based intrusion detection system. By combining real-time packet capture, advanced feature engineering, and state-of-the-art machine learning algorithms, the system aims to provide accurate, scalable, and practical network intrusion detection capabilities.

1.2 MOTIVATION

The escalating cybersecurity threat landscape serves as the primary impetus for this project. Recent data breaches, ransomware attacks, and state-sponsored cyber operations have demonstrated the critical importance of robust intrusion detection capabilities. The financial and reputational costs associated with successful network intrusions—ranging from data theft and operational disruption to compliance violations and loss of customer trust—highlight the urgent need for effective countermeasures.

From a technical perspective, this project is motivated by the opportunity to bridge the gap between academic research in machine learning and practical, deployable network security solutions. While extensive literature exists on ML-based intrusion detection, few systems address the complete pipeline from raw packet capture to real-time classification, coupled with accessible user interfaces and operational tooling. This creates an opportunity to develop an end-to-end system suitable for deployment in real-world environments.

Furthermore, the availability of high-quality, publicly available network datasets (such as CICIDS2017) combined with open-source frameworks enables the development of sophisticated IDS systems without prohibitive infrastructure costs. This democratization of advanced security tools aligns with the mission of making enterprise-grade detection capabilities accessible to organizations of all sizes.

The motivation is also driven by pedagogical considerations—developing this system provides valuable insights into network security fundamentals, machine learning model training and deployment, distributed system design, and full-stack application development. The project represents an opportunity to synthesize knowledge across multiple domains and contribute meaningful insights to the cybersecurity community.

1.3 PROBLEM DEFINITION

In the contemporary digital ecosystem, network intrusions represent a persistent and evolving threat to organizations of all sizes and sectors. The fundamental problem addressed by this project is the critical challenge of accurately and efficiently detecting malicious network activities amidst the overwhelming volume, complexity, and diversity of modern network traffic.

The core challenge manifests in several interconnected ways. First, traditional signature-based Intrusion Detection Systems (IDS) operate on the principle of matching network traffic against known attack signatures, making them inherently reactive. By the time a new attack variant or zero-day vulnerability is discovered and signatures are updated, malicious actors may have already compromised multiple systems. This reactive approach leaves significant security gaps, particularly against advanced persistent threats (APTs) and sophisticated threat actors who continuously evolve their tactics, techniques, and procedures (TTPs) to evade detection.

Second, the sheer volume of network traffic in modern enterprises makes manual threat detection and analysis impractical. Security operations centers (SOCs) face alert fatigue—systems generate thousands of alerts daily, many of which are false positives. Analysts struggle to distinguish genuine threats from benign anomalies, leading to delayed incident response, missed attacks, and inefficient resource utilization. The signal-to-noise ratio in traditional IDS outputs is often prohibitively high, rendering them less effective in practice.

Third, modern attacks employ sophisticated techniques to obfuscate their nature and blend with legitimate traffic. Encrypted traffic, polymorphic malware, low-and-slow attack patterns, and application-layer attacks circumvent traditional network-level detection mechanisms. Additionally, insider threats and compromised legitimate accounts generate primarily benign-appearing traffic, making rule-based detection particularly ineffective.

The consequences of inadequate network intrusion detection are far-reaching. Successful network breaches result in data theft, operational disruption, financial losses, regulatory compliance violations, and damage to organizational reputation. For critical infrastructure sectors such as energy, healthcare, and finance, the consequences can be catastrophic, affecting not only the organization but broader societal systems. The average time to detect a breach has been measured in months or years, during which attackers maintain persistence and expand their foothold.

Furthermore, many organizations, particularly small and medium-sized enterprises (SMEs), lack the expertise and resources to deploy and maintain sophisticated security infrastructure, leaving them particularly vulnerable to network-based attacks.

Therefore, the problem this project seeks to address is the need for an intelligent, adaptive, scalable, and accessible system capable of detecting network intrusions in real-time with high accuracy and minimal false positives, independent of predefined attack signatures, while remaining practical for deployment in diverse organizational contexts.

1.4 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design, implement, and deploy a production-grade machine learning-based Network Intrusion Detection System (NIDS) capable of accurately identifying and classifying network intrusions in real-time. The system aims to address the limitations of signature-based IDS by leveraging machine learning algorithms to detect both known and novel attack patterns through anomaly detection and behavioral analysis. By combining live packet capture, advanced feature engineering, and intelligent classification, the project seeks to establish a practical, deployable solution that enhances network security for organizations.

Specific technical objectives include: (i) implementing a multi-threaded packet capture and processing pipeline using Scapy for real-time network traffic analysis; (ii) engineering and extracting 51-dimensional feature vectors from network flows to effectively represent network behavior; (iii) training and evaluating a Random Forest classifier on the CICIDS2017 dataset to achieve high accuracy in distinguishing benign traffic from various attack classes (DoS, DDoS, Brute Force, Web Attacks, Infiltration, and Botnet); (iv) developing a comprehensive REST API with 12 endpoints for system control, data retrieval, and integration with external security frameworks; and (v) creating an interactive web-based dashboard with real-time visualization of network statistics, active flows, and security events.

A critical objective is to ensure the system is practical and accessible for real-world deployment. This involves developing a cross-platform command-line interface (CLI) tool for system management, creating comprehensive documentation, and designing the architecture for easy integration into existing security infrastructure. The system should support flexible configuration, handle high-volume network traffic efficiently, and minimize latency in threat detection and response.

Finally, the project aims to establish a foundation for future research and development in intelligent network security. By successfully demonstrating the effectiveness of machine learning-based intrusion detection, documenting the complete implementation pipeline, and providing accessible tools and APIs, the project contributes valuable insights and practical guidance to the cybersecurity community and enables organizations to adopt intelligent threat detection capabilities.

1.5 LIMITATIONS OF THE PROJECT

While this project aims to establish a comprehensive machine learning-based framework for network intrusion detection, it is important to acknowledge several inherent limitations in its current scope and implementation:

Data And Dataset Limitations

- **Static Dataset Evaluation:** The model is trained and evaluated on the CICIDS2017 dataset, which is static and represents network traffic patterns from 2017. The dynamic nature of cyber threats, with continuously evolving attack methodologies and traffic patterns, poses challenges for models trained on historical data. Zero-day exploits and novel attack vectors may not be adequately represented in the training dataset.
- **Dataset Time Gap:** The significant time gap between the training dataset (2017) and current operational deployment may result in distribution shifts and reduced model generalization to contemporary attack patterns.
- **Limited Attack Diversity:** While CICIDS2017 includes multiple attack classes (DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet), it may not comprehensively cover all attack categories or sophisticated hybrid attacks that combine multiple techniques.

Technical and Architectural Limitations

- **Single Interface Capture:** The system captures traffic from a single network interface, limiting its applicability in complex, multi-interface enterprise networks or environments requiring distributed monitoring across multiple network segments.
- **Unencrypted Traffic Focus:** The feature extraction methodology relies on packet-level and flow-level statistics. Modern network traffic is increasingly encrypted (HTTPS, TLS, VPN), limiting visibility into application-layer attack patterns and payload-based anomalies. The system cannot inspect encrypted traffic payloads without decryption capabilities.

- **Scalability Constraints:** While designed for real-time processing, the system's performance on extremely high-volume networks (>100K packets/second) requires validation. Computational overhead may impact detection latency or require horizontal scaling infrastructure.
- **Feature Engineering Dependency:** The 51-dimensional feature set is manually engineered and fixed. Novel attack patterns that manifest through previously unseen combinations of features may not be captured effectively.

Model-Related Limitations

- **Single Classifier Architecture:** The system relies exclusively on Random Forest for classification. While Random Forest provides good baseline performance, other ensemble methods, deep learning architectures (LSTMs, CNNs, Transformers), or hybrid approaches might offer superior performance and are not extensively explored in this implementation.
- **Class Imbalance:** Real-world network traffic exhibits significant class imbalance (benign traffic vastly outnumbers malicious traffic). While handled during training, residual class imbalance effects may impact minority class detection accuracy.
- **No Concept Drift Adaptation:** The trained model is static; it does not adapt to gradual changes in network behavior or emerging attack patterns without retraining. Continuous learning mechanisms are not implemented.

Operational Limitations

- **No Decryption or Deep Packet Inspection:** The system does not perform deep packet inspection or payload analysis, limiting its ability to detect application-layer attacks or malicious payloads hidden within encrypted traffic.
- **Absence of Threat Intelligence Integration:** The system does not incorporate external threat intelligence feeds, known malicious IPs/domains, or behavioral intelligence from security communities, which could enhance detection capabilities.
- **Limited Contextual Information:** Predictions are based solely on network flow characteristics. External context such as geolocation, time-of-day patterns, user identity, or asset criticality is not incorporated into the classification process.

- **Real-Time Response Limitations:** The system provides detection and alerting but does not implement automated response mechanisms such as traffic blocking, connection termination, or integration with firewall/SIEM systems for coordinated incident response.

Deployment and Usability Limitations

- **Linux-Centric Implementation:** While the CLI tool is cross-platform, packet capture via Scapy is primarily optimized for Linux environments. Windows and macOS support may exhibit reduced performance or require additional configuration.
- **No Role-Based Access Control:** The web dashboard and API lack user authentication and role-based access control (RBAC), limiting deployment in multi-user enterprise environments requiring security boundaries.
- **Limited Historical Data Analysis:** The system focuses on real-time detection and does not provide comprehensive historical analysis, trend detection, or long-term threat pattern analysis over extended time periods.

Research and Validation Constraints:

- **Benchmarking on Limited Datasets:** Evaluation is confined to CICIDS2017. Validation on other public datasets (NSL-KDD, UNSW-NB15) or proprietary enterprise traffic is not performed, limiting generalization claims.
- **Absence of Adversarial Robustness Testing:** The system has not been evaluated against adversarial attacks or evasion techniques specifically designed to fool machine learning-based IDS systems.
- **No Comparison with State-of-the-Art Systems:** Performance comparison with contemporary commercial IDS solutions or recent academic implementations is not conducted.

Despite these limitations, the project successfully demonstrates the feasibility and practical utility of machine learning-based network intrusion detection and establishes a solid foundation for future enhancements and extensions.

1.6 ORGANIZATION OF THE PROJECT

This document is structured to provide a comprehensive exposition of the "**NETWORK INTRUSION DETECTION SYSTEM (NIDS)**" project, detailing the research, design, implementation, and evaluation of the machine learning-based intrusion detection framework. The subsequent sections guide the reader through various aspects of this work in a logical and progressive manner.

Chapter 2: System Specifications outlines the technical environment and infrastructure in which the project was developed and operates. Section 2.1 details the software specifications, including programming languages (Python), frameworks (Flask, Scapy), libraries (scikit-learn, pandas, NumPy), and development tools employed throughout the project. Section 2.2 describes the hardware specifications, including processor, memory, storage, and network interface requirements for optimal system performance.

Chapter 3: Literature Survey provides a critical review of existing scholarly work and systems relevant to network intrusion detection, cybersecurity, and machine learning applications in security. Section 3.1 serves as an introduction to contemporary approaches in network security. Section 3.2 examines existing IDS systems and their methodologies, highlighting both signature-based and machine learning-based approaches. Section 3.3 identifies the limitations and shortcomings of existing systems, justifying the need for the proposed solution. Section 3.4 introduces the NIDS system developed in this project, positioning it contextually within the reviewed literature.

Chapter 4: Design and Implementation details both the architectural design and complete implementation of the proposed NIDS system. Section 4.1 provides an introduction to design considerations and overall system architecture. Section 4.2 presents a comprehensive system architecture diagram illustrating the interaction between major components (packet capture, flow aggregation, feature extraction, classification, and output). Section 4.3 describes the data flow and processing pipeline. Section 4.4 outlines the machine learning model architecture, feature engineering methodology, and classification approach. Section 4.5 details the API design and web dashboard interface. Section 4.6 describes the implementation process, including the core pipeline architecture, multi-threading design, component integration, and selected source code excerpts highlighting key implementations.

Chapter 5: Testing, Results, and Conclusion presents the testing methodology, validation results, key findings, and future directions. Section 5.1 describes the testing approach, including unit tests, integration tests, and system-level validation. Section 5.2 presents performance metrics, classification accuracy, precision, recall, F1-scores, and confusion matrices, along with visualizations from the web dashboard. Section 5.3 summarizes the project's key findings and the extent to which objectives were achieved. Section 5.4 discusses potential areas for future work, including model improvements, feature enhancements, scalability considerations, and integration with enterprise security frameworks.

SYSTEM SPECIFICATIONS

2. SYSTEM SPECIFICATIONS

The Network Intrusion Detection System requires a well-defined set of functional and non-functional requirements to ensure effective operation in real-world network environments. The following subsections outline these requirements, along with the specific software and hardware specifications necessary for deployment.

2.1 SOFTWARE SPECIFICATIONS

2.1.1 FUNCTIONAL REQUIREMENTS

The system must capture live network traffic in real-time and extract meaningful features for classification. It shall classify network activities as benign or malicious (DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet). The system must provide a REST API for control and data retrieval, maintain prediction history, and display real-time statistics through a web dashboard. A CLI tool must enable system management across multiple platforms.

2.1.2 NON-FUNCTIONAL REQUIREMENTS

The system must achieve detection accuracy exceeding 90% and handle high-volume traffic without significant latency. The architecture must be modular and maintainable, with cross-platform compatibility. The API must respond within acceptable timeframes, and the system must implement security measures against input attacks. Configuration must persist across application restarts.

2.1.3 PROGRAMMING LANGUAGES AND CORE LIBRARIES

Python 3.10+ is the primary programming language. Scapy handles packet capture and network traffic analysis. Scikit-learn provides the Random Forest classifier for attack detection. NumPy and Pandas manage numerical computations and data handling. Click enables cross-platform command-line functionality. JobLib serializes and persists the trained model.

2.1.4 BACKEND AND API FRAMEWORK

- Flask provides the REST API framework and web server for backend operations
- Flask-based HTTP endpoints handle system control, data retrieval, and status queries
- JSON serialization for API responses and configuration management
- WSGI-compatible deployment on development and production servers

2.1.5 FRONTEND AND DASHBOARD TECHNOLOGIES

- HTML5 for markup structure of the web dashboard

- CSS3 for responsive styling and grid-based layout
- JavaScript (ES6+) for real-time AJAX requests and dynamic DOM updates
- No external frontend frameworks (jQuery, React, Vue) - vanilla JavaScript for minimal dependencies

2.1.6 DEVELOPMENT AND DEPLOYMENT STACK

- VS Code serves as the development environment with Python debugging capabilities
- Git manages version control and collaborative development
- Python's threading module for concurrent operations

2.2 HARDWARE SPECIFICATIONS

2.2.1 MINIMUM REQUIREMENTS

- Processor: Dual-core 2.0 GHz or higher
- RAM: 4 GB minimum
- Storage: 2 GB free disk space for application and dependencies
- Network: Standard Ethernet interface with promiscuous mode support

2.2.2 RECOMMENDED SPECIFICATIONS

- Processor: Quad-core 2.5 GHz or above (Intel Core i5 or equivalent)
- RAM: 8 GB for production deployment
- Storage: Additional capacity for prediction history and logs
- Display: 1024×768 or higher resolution for web dashboard access

2.2.3 OPERATING SYSTEM COMPATABILITY

- Linux (Ubuntu 18.04+, Debian 10+) - Primary platform
- Windows 10/11 with Python environment
- macOS with Homebrew package management

LITERATURE SURVEY

3. LITERATURE SURVEY

3.1 INTRODUCTION

Network security has evolved significantly over the past two decades, with intrusion detection emerging as a critical research area in cybersecurity. Traditional approaches relied on signature-based detection, matching network traffic against known attack patterns maintained in rule databases. However, the increasing sophistication of cyber attacks, the emergence of zero-day vulnerabilities, and the constant evolution of attack methodologies have exposed fundamental limitations in these reactive approaches.

The application of machine learning to network intrusion detection represents a paradigm shift, enabling systems to learn attack patterns from historical data and identify anomalies without predefined signatures. This survey examines existing approaches, their strengths and limitations, and positions the proposed NIDS system within the broader context of contemporary network security research.

3.2 EXISTING SYSTEMS

3.2.1 SIGNATURE-BASED INTRUSION DETECTION SYSTEMS

Traditional IDS solutions such as Snort and Suricata operate on signature-based detection principles. These systems maintain extensive rule databases defining known attacks through pattern matching on network traffic. When packets match predefined signatures, alerts are generated for human review. While these systems have proven effective for detecting known threats, they suffer from significant limitations in modern threat landscapes.

Signature-based approaches require constant rule updates to address emerging threats, creating a reactive detection model. Zero-day vulnerabilities—previously unknown exploits—cannot be detected by definition, as no signatures exist. Additionally, attackers actively develop evasion techniques such as encryption, traffic obfuscation, and polymorphic malware to circumvent signature detection. The high false-positive rate in production networks leads to alert fatigue, overwhelming security operations teams and reducing their effectiveness.

3.2.2 MACHINE LEARNING-BASED APPROACHES

Recent research has explored machine learning and statistical methods for anomaly-based intrusion detection. Early work applied classical algorithms such as Support Vector Machines (SVM), Naïve Bayes, and Decision Trees to network traffic classification. These approaches extract

statistical features from network flows—such as packet counts, byte volumes, protocol distributions, and temporal characteristics—and train classifiers to distinguish benign from malicious traffic.

Studies have demonstrated that Random Forest ensembles achieve strong performance on benchmark datasets like CICIDS2017 and NSL-KDD, often exceeding 90% accuracy in attack detection. Random Forest's ability to handle high-dimensional feature spaces, manage non-linear relationships, and provide feature importance rankings makes it particularly suitable for network traffic analysis. The algorithm is computationally efficient for inference, enabling real-time classification of high-volume traffic.

Ensemble methods combine multiple weak learners to improve prediction robustness. Gradient Boosting approaches like XGBoost have shown promise in capturing complex attack patterns through sequential decision tree refinement. These methods naturally handle feature interactions and provide interpretability through feature importance analysis—a critical requirement for security operations who must understand why specific flows are classified as attacks.

3.2.3 DEEP LEARNING AND NEURAL NETWORK APPROACHES

Recent advances have applied deep learning architectures to intrusion detection. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks capture sequential dependencies in network traffic, modeling temporal patterns of attacks. Convolutional Neural Networks (CNNs) extract spatial patterns from traffic features, while autoencoders identify unusual patterns through unsupervised learning.

While deep learning approaches can achieve high accuracy on benchmark datasets, they introduce significant computational overhead, require substantial training data, and present interpretability challenges. Deployment of neural network-based IDS in production environments encounters practical constraints including inference latency, memory requirements, and the need for GPU acceleration. Additionally, deep learning models are prone to adversarial attacks and may exhibit poor generalization to network traffic distributions different from training data.

3.2.4 HYBRID AND CONTEXT-AWARE APPROACHES

Contemporary research explores hybrid systems combining signature-based and machine learning approaches. These systems leverage rule-based detection for known threats while employing ML models to identify anomalies and novel attacks. Context-aware detection

incorporates external information such as threat intelligence feeds, network topology, asset criticality, and user behavior baselines to enhance classification accuracy.

Multi-stage detection pipelines progressively refine predictions, combining fast signature matching with computationally expensive ML models only when necessary. This approach balances detection accuracy with operational efficiency. However, integration of external context requires sophisticated data fusion and often demands enterprise-scale infrastructure and security intelligence platforms.

3.2.5 DATASET AND BENCHMARK CONSIDERATIONS

The CICIDS2017 dataset has become a standard benchmark for IDS evaluation, providing labeled network flows with both benign and attack traffic across multiple categories (DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet). Other datasets such as NSL-KDD and UNSW-NB15 provide alternative benchmarks, though each exhibits different characteristics and limitations.

Researchers have noted that models trained on one dataset often exhibit reduced performance on others, indicating potential overfitting to dataset-specific characteristics. The temporal dynamics of cyber attacks—where attack patterns evolve and new methodologies emerge—present ongoing challenges for static models trained on historical data.

3.3 DISADVANTAGES OF EXISTING SYSTEMS

3.3.1 LIMITATIONS OF SIGNATURE-BASED SYSTEMS

Signature-based IDS solutions cannot detect zero-day exploits or previously unknown attack variants, as attack signatures must be manually defined and distributed before detection can occur. This reactive model inherently lags behind threat evolution. Attackers actively develop evasion techniques such as traffic encryption, payload obfuscation, and protocol manipulation to circumvent signature matching. Maintaining current rule databases requires significant human expertise and continuous updates, scaling poorly as attack complexity increases.

The high false-positive rate in production networks leads to alert fatigue, where security operators become desensitized to alarms and overlook genuine threats. Signature-based systems often trigger alerts on normal network anomalies unrelated to security, wasting analyst time on triage and investigation of non-threatening events.

3.3.2 COMPUTATIONAL AND DEPLOYMENT COMPLEXITY OF DEEP LEARNING

Deep learning approaches, while achieving high accuracy on benchmark datasets, introduce substantial computational overhead. LSTM and transformer-based models require GPU acceleration for practical inference speeds. This hardware dependency increases deployment costs and complexity, restricting deployment to organizations with sufficient infrastructure budgets. The inference latency of complex neural networks may exceed acceptable thresholds for real-time detection in high-volume networks.

Deep learning models exhibit poor interpretability—security operations teams cannot easily understand why specific predictions were made, complicating incident response and threat analysis. The "black box" nature of neural networks raises concerns in regulated environments where explainability is required for compliance.

3.3.3 DATA AND GENERALIZATION CHALLENGES

Existing IDS systems demonstrate poor generalization across different network environments and traffic distributions. Models trained on benchmark datasets often underperform when deployed on novel network traffic with different characteristics, a phenomenon known as dataset shift or distribution mismatch. Attackers intentionally alter their tactics to evade deployed systems, rendering models trained on historical data increasingly ineffective over time.

The availability of representative training data remains a significant barrier. Obtaining labeled network datasets capturing legitimate attack diversity while maintaining privacy is challenging. Most public datasets, such as CICIDS2017, are now several years old, creating concern that attack patterns represented in training data may no longer reflect contemporary threats.

3.3.4 INTEGRATION AND OPERATIONAL LIMITATIONS

Existing IDS platforms often operate in isolation, providing detection without integration into broader security ecosystems. Integration with SIEM systems, ticketing platforms, and automated response mechanisms requires custom development and maintenance. Commercial IDS solutions often lack flexibility for customization, while open-source alternatives require significant engineering effort to deploy and secure.

The high cost of commercial IDS solutions and supporting infrastructure (monitoring, management, incident response) limits accessibility for smaller organizations and institutions. Manual alert investigation and threat analysis create bottlenecks in security operations, particularly for SOCs handling high-volume alerts.

3.3.5 SCALABILITY AND PRACTICAL DEPLOYMENT

Many IDS solutions struggle to handle modern network traffic volumes. High-speed networks generating millions of packets per second can overwhelm detection systems, forcing trade-offs between coverage and performance. Distributed IDS deployments across multiple network segments introduce coordination and management complexity.

Encrypted traffic—increasingly dominant in modern networks—creates a fundamental challenge for packet-level analysis. Many contemporary attacks operate within encrypted HTTPS sessions, rendering packet payload inspection impossible without decryption infrastructure. Network monitoring on encrypted traffic requires either decryption proxies (introducing performance penalties and privacy concerns) or reliance on metadata analysis (limited detection capabilities).

3.3.6 ABSENCE OF CONTEXTUAL INTELLIGENCE

Existing systems typically cannot incorporate organizational context such as asset criticality, user roles, threat intelligence feeds, or business processes. Detection decisions based solely on network features miss important contextual signals that distinguish legitimate anomalies from actual security threats. Integration of threat intelligence from external sources requires manual processes or complex middleware, limiting the timeliness of threat incorporation.

3.4 PROPOSED SYSTEM

3.4.1 SYSTEM OVERVIEW AND ARCHITECTURE

The proposed Network Intrusion Detection System (NIDS) addresses the limitations of existing approaches by combining practical machine learning implementation with accessible deployment and operational tooling. Rather than pursuing complex deep learning architectures requiring specialized hardware, the system leverages Random Forest classification on engineered network flow features, delivering high accuracy with minimal computational overhead and interpretable results.

The NIDS architecture comprises three integrated components: a network packet capture and processing pipeline, a machine learning classification module, and a comprehensive operational interface including REST API and web dashboard. This modular design enables independent development, testing, and deployment of each component while maintaining system coherence.

3.4.2 REAL-TIME PACKET CAPTURE AND FEATURE EXTRACTION

The system employs Scapy for live network packet capture, monitoring specified network interfaces and extracting individual packets in real-time. Rather than analyzing raw packets individually, NIDS aggregates packets into network flows—bidirectional sequences of packets sharing common source/destination IP addresses and ports—providing a higher-level representation of network activity.

From each flow, the system extracts 51-dimensional feature vectors capturing statistical characteristics: packet counts, byte volumes, packet length statistics, inter-arrival times, protocol distributions, and TCP flag frequencies. These features represent meaningful abstractions of network behavior, enabling machine learning models to recognize patterns distinguishing attacks from normal traffic. Feature engineering follows established methodologies from network traffic analysis research while maintaining computational efficiency for real-time processing.

3.4.3 MACHINE LEARNING CLASSIFICATION

The NIDS employs a Random Forest classifier trained on the CICIDS2017 dataset, capable of distinguishing benign traffic from six attack categories: Denial of Service (DoS), Distributed Denial of Service (DDoS), Brute Force attacks, Web Attacks, Infiltration, and Botnet traffic. Random Forest was selected for its strong performance on network traffic classification, computational efficiency enabling real-time inference, feature importance capability supporting security analyst interpretation, and robustness to feature scaling variations.

Unlike signature-based systems, the trained model captures statistical patterns inherent to different attack types, enabling detection of attack variants and novel methodologies not explicitly present in training data. The classification model produces both discrete predictions (attack class) and confidence scores, enabling practitioners to set detection thresholds balancing sensitivity and specificity for their operational context.

3.4.4 OPERATIONAL INTERFACE AND ACCESSIBILITY

The system provides a REST API with twelve endpoints enabling programmatic control, data retrieval, and integration with external security tools. API operations include starting/stopping traffic capture, querying system status, retrieving historical predictions, accessing active flow information, and obtaining statistical summaries. This API enables automation of incident response workflows, integration with Security Information and Event Management (SIEM) platforms, and development of custom operational tools.

A web-based dashboard provides real-time visualization of network security events without requiring specialized software installation. The dashboard displays active statistics (total predictions, attack/benign counts, attack rate distribution), recent predictions with confidence scores, active network flows, and system configuration. Real-time updates occur via AJAX polling, providing operators immediate visibility into network incidents.

3.4.5 CROSS-PLATFORM MANAGEMENT AND DEPLOYMENT

A command-line interface (CLI) tool enables system management across Linux, Windows, and macOS platforms. CLI commands support configuration management, background daemon execution, log retrieval, and process control. This accessibility reduces deployment barriers and enables administrators to integrate NIDS into diverse operational environments.

The system implements a multi-threaded architecture where packet capture, flow aggregation, and classification operations execute concurrently, enabling efficient processing of high-volume traffic without blocking individual components.

3.4.6 ADDRESSING EXISTING SYSTEM LIMITATIONS

The NIDS design explicitly addresses limitations identified in contemporary systems. By employing machine learning rather than signatures, the system detects novel attacks and variants without requiring rule updates. The engineered feature approach provides computational efficiency suitable for real-time deployment without GPU requirements. Interpretable Random Forest models enable security operators to understand classification decisions, supporting incident investigation and threat analysis.

The practical, lightweight architecture reduces deployment barriers compared to complex enterprise platforms, enabling adoption by organizations with limited infrastructure budgets. The integrated REST API and CLI tooling facilitate operational integration and automation. Real-time visualization of network security events supports situational awareness and timely incident response.

While the system focuses on network flow analysis rather than encrypted payload inspection, this approach proves robust across encrypted traffic by analyzing connection metadata and traffic patterns. The modular architecture enables future enhancements such as integration of external threat intelligence, adoption of more sophisticated ML models, and expansion to distributed deployment across multiple network segments.

DESIGN AND IMPLEMENTATION

4. DESIGN AND IMPLEMENTATION

4.1 INTRODUCTION

The design of the Network Intrusion Detection System represents a careful balance between academic rigor and practical applicability. This chapter details the architectural decisions, component design, and implementation strategies employed to create a functional, deployable intrusion detection system. The design philosophy prioritizes modularity, enabling independent development and testing of components while maintaining system coherence through well-defined interfaces and data contracts.

A critical design principle is the separation of concerns, where packet capture and processing, machine learning classification, and operational interfaces operate as distinct modules. This separation enables teams to work in parallel, facilitates unit testing at the component level, and allows future enhancements to individual modules without requiring complete system redesign.

The system architecture encompasses three primary layers: the data acquisition and processing pipeline responsible for real-time network traffic capture and feature extraction, the machine learning classification layer utilizing pre-trained models for threat detection, and the operational interface layer providing multiple mechanisms for system control and result visualization. Each layer maintains clear input/output contracts, allowing independent scalability and technology upgrades.

Implementation decisions prioritize practical deployment over theoretical perfection. Rather than pursuing state-of-the-art deep learning architectures requiring substantial computational resources, the system employs Random Forest classification on engineered features, delivering interpretable results with minimal hardware requirements. This pragmatic approach acknowledges the trade-off between model complexity and operational practicality, ensuring the system remains accessible to organizations with diverse infrastructure capabilities.

The multi-threaded architecture enables concurrent execution of packet capture, flow aggregation, classification, and API request handling, preventing any single operation from blocking others and ensuring responsive performance under load. Careful synchronization mechanisms protect shared resources while minimizing lock contention that would degrade throughput.

This chapter first presents the overall system architecture, then examines each major component in detail, discusses the data flow and processing pipeline, describes key implementation patterns, and finally explores the design decisions underlying critical choices in the system's development.

4.2 SYSTEM ARCHITECTURE

4.2.1 OVERVIEW

The NIDS system architecture comprises three integrated layers working in concert to capture network traffic, identify threats, and present results to operators.

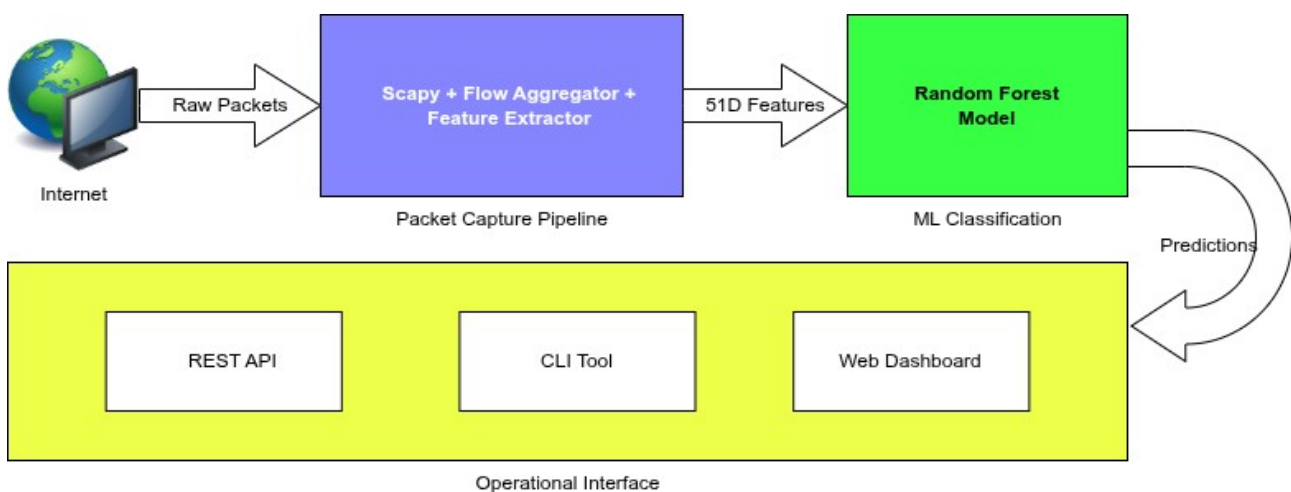


Figure 4.2.1.1: Architecture Diagram

As shown in Figure 4.2.1.1, the flow diagram illustrates the high-level architecture, showing the flow of data from raw network packets through classification to operational outputs. The modular design enables each layer to scale and evolve independently while maintaining well-defined interfaces between components.

4.2.2 DATA ACQUISITION AND PROCESSING PIPELINE

The first layer handles real-time network traffic capture and transformation into structured features suitable for machine learning classification. This pipeline begins with Scapy, which intercepts packets from the specified network interface operating in promiscuous mode. Rather than analyzing individual packets, the system aggregates packets into bidirectional network flows—sequences of packets sharing common source and destination addresses and ports.

Flow aggregation serves multiple purposes: it reduces dimensionality by grouping related packets, provides a meaningful representation of network activity aligning with how security

professionals conceptualize network behavior, and enables efficient timeout-based cleanup of inactive flows. Once flows are established, the system extracts 51-dimensional feature vectors capturing statistical characteristics including packet counts, byte volumes, inter-arrival times, protocol distributions, and TCP flag patterns.

4.2.3 MACHINE LEARNING AND CLASSIFICATION LAYER

The second layer accepts feature vectors from the pipeline and applies the trained Random Forest classifier to produce predictions. This layer maintains the trained model in memory, loaded at system startup from a persisted joblib file. Classification produces two outputs: a discrete prediction (benign or one of six attack classes) and a confidence score representing the model's certainty.

The ML layer is designed as a stateless component—given identical input features, it always produces identical predictions. This property simplifies scaling to distributed architectures and enables consistent behavior across system restarts. The layer maintains no history or context; all decision logic resides in the trained model.

4.2.4 OPERATIONAL INTERFACE LAYER

The third layer provides multiple interfaces for system control, result retrieval, and visualization. The REST API exposes twelve endpoints enabling programmatic access to system functionality, querying predictions, managing configuration, and triggering control operations. The API is stateless and synchronous, with each request independently resolved without depending on previous requests or global application state.

The web dashboard provides real-time visualization of network security events, displaying active statistics, recent predictions, and flow information through a single-page application architecture. The CLI tool enables local system management including configuration, daemon control, and log retrieval. All three interfaces operate against shared backend state (predictions database, configuration files, model) but maintain independence in their implementation.

4.2.5 DATA FLOW INTERACTION

Data flows through the system in two primary modes: the forward path where network packets become classifications, and the query path where external clients retrieve results through the API or dashboard.

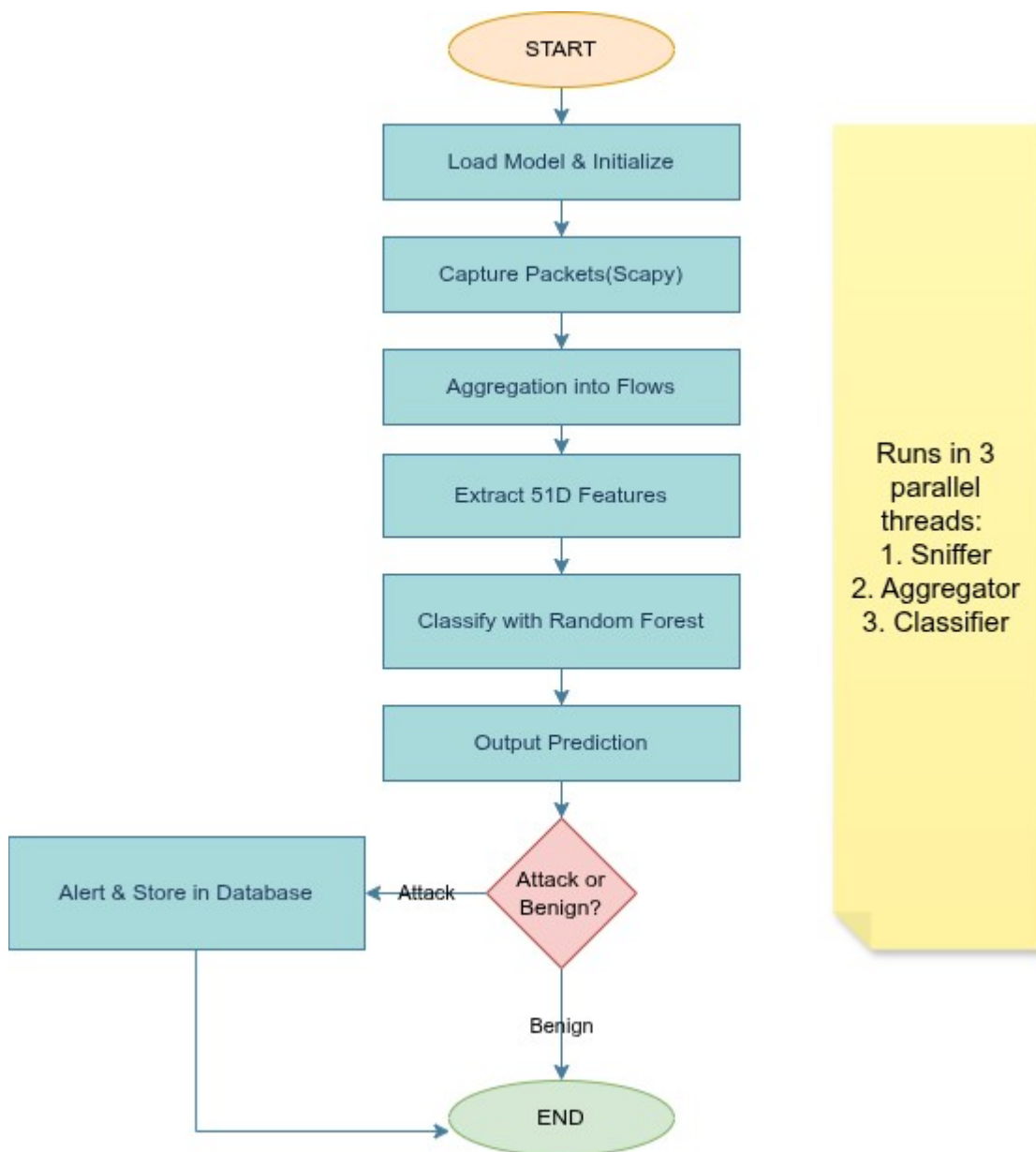


Figure 4.2.5.1: Data Acquisition And Processing Pipeline

As shown in **Figure 4.2.5.1**, in the forward path, the packet capture pipeline continuously acquires packets, aggregates them into flows, and extracts features. Once sufficient packets are collected (or timeout occurs), features flow to the classification layer where predictions are generated. Predictions are immediately persisted to the database and made available through the operational interface.

In the query path, external clients request information through the REST API or web dashboard. These queries execute independently of the forward processing path, retrieving pre-

computed results from the database. This separation ensures that query operations never block packet processing, maintaining detection latency below operational thresholds.

4.2.6 THREADING MODEL

The architecture employs three concurrent threads executing the packet capture, flow aggregation, and classification operations. The sniffer thread continuously captures packets and queues them for processing. The aggregator thread dequeues packets, maintains flow state, and identifies completed flows for feature extraction. The classifier thread dequeues completed flows, extracts features, applies the model, and persists predictions.

This multi-threaded design prevents any single operation from blocking others. If classification temporarily slows (perhaps due to system load), the sniffer and aggregator continue operating, allowing packets to queue without loss. Similarly, if packet capture momentarily stalls, aggregation and classification continue processing buffered data. Thread synchronization employs thread-safe queues minimizing lock contention.

4.2.7 CONFIGURATION AND STATE MANAGEMENT

The system maintains persistent configuration in a JSON file enabling settings to survive application restarts. Configuration includes network interface selection, API port, model directory location, and timeout parameters. State—including active flows and predictions—is managed in memory during operation and persisted to SQLite database for historical retrieval.

This architecture distinguishes between configuration (persistent, infrequently changing) and state (dynamic, changing continuously). Separating these concerns simplifies both development and operations, allowing configuration to be version-controlled while state can be aggressively cached for performance.

4.3 COMPONENT DESIGN

4.3.1 PACKET CAPTURE MODULE

The packet capture module forms the foundation of the data acquisition pipeline, responsible for intercepting network traffic and converting raw packets into structured data. This module uses Scapy, a Python library providing packet manipulation and sniffing capabilities across multiple platforms without requiring specialized kernel-level drivers or system privileges on Linux systems.

As shown in **Figure 4.3.1.1**, the module operates by configuring Scapy to capture from a specified network interface in promiscuous mode, allowing it to observe all traffic on the network

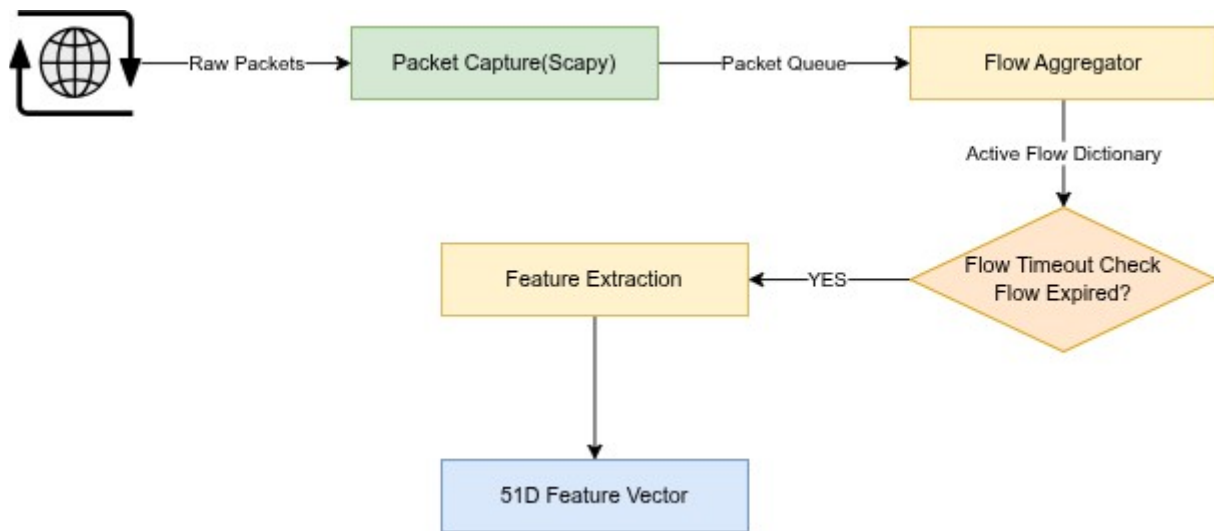


Figure 4.3.1.1: Packet Capture Flow

segment regardless of destination address. Promiscuous mode is essential for detecting traffic flows not directly destined for the monitoring system. The sniffer runs in a dedicated thread, extracting packets continuously and immediately passing them to the flow aggregation module through a thread-safe queue.

For each captured packet, the module extracts basic information including source/destination IP addresses, ports, protocol type (TCP/UDP), and packet size. This extraction occurs at capture time to minimize subsequent processing burden. The packet dissection leverages Scapy's protocol layers, automatically parsing IP, TCP, and UDP headers. Packets are discarded if they lack necessary layer information (e.g., IP layer missing) rather than buffering incomplete packets.

The module maintains configurable timeout and buffer thresholds. If the input queue exceeds a maximum size (indicating downstream bottleneck), the oldest packets are selectively dropped to prevent unbounded memory growth. This design choice prioritizes recent packets over historical ones, assuming recent activity is more relevant to current security state.

4.3.2 FLOW AGGREGATION AND TIMEOUT

Network flows represent bidirectional sequences of packets sharing common source and destination addresses and ports. The flow aggregation module transforms individual packets into higher-level flow representations, providing semantically meaningful units for feature extraction and classification.

The module maintains a dictionary of active flows, keyed by a 5-tuple (source IP, destination IP, protocol, source port, destination port). When a packet arrives, the aggregator looks up its corresponding flow in the dictionary. If the flow exists, packet information (count, byte volume, timestamp) is updated. If not found, a new flow entry is created with initial packet metadata.

A critical design aspect is flow timeout—determining when a flow should be considered complete and eligible for feature extraction and classification. The module implements both inactivity timeout (flows inactive for 120 seconds expire) and hardcoded session timeout (all flows expire after 300 seconds regardless of recent activity). Expired flows are immediately processed for feature extraction, enabling timely classification. This dual timeout mechanism balances timeliness (not waiting indefinitely for new packets) with statistical accuracy (having sufficient data for reliable feature calculation).

When flows expire, they transition to a processing queue for feature extraction. Concurrently, the aggregator continues accepting new packets and maintaining active flows, preventing timeout processing from blocking packet aggregation.

4.3.3 FEATURE EXTRACTION

Feature extraction transforms raw flow data into 51-dimensional numerical vectors suitable for machine learning classification. Each dimension captures a statistical or structural characteristic of the flow, derived from packet-level observations accumulated during the flow's lifetime.

The extracted features span multiple categories: volume statistics (total packets, total bytes, minimum/maximum/mean packet sizes), temporal characteristics (flow duration, average inter-arrival time, standard deviation of inter-arrival times), protocol-level features (TCP flag distributions, protocol ratios if mixed-protocol flows), and rate-based metrics (packets per second, bytes per second).

Feature extraction employs numpy for efficient vectorized operations. For each expired flow, the module computes all 51 dimensions, handling edge cases where flows contain insufficient data (e.g., single-packet flows). Features are normalized to zero-mean unit variance using pre-computed normalization parameters derived during model training, ensuring consistency between training and inference.

The extracted feature vector is immediately available for classification with no intermediate processing. Features are immutable once computed, enabling safe concurrent access from the classification thread without synchronization overhead.

4.3.4 ML CLASSIFICATION MODULE

The classification module applies the pre-trained Random Forest model to feature vectors, producing predictions and confidence scores. This module loads the model from disk during system initialization, maintaining it in memory for rapid inference.

For each feature vector received, the module passes it to the Random Forest model using scikit-learn's `predict` and `predict_proba` methods. The `predict` method returns the predicted class (benign or one of six attack categories: DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet). The `predict_proba` method returns probability estimates for all classes, from which the maximum probability (confidence score) is extracted.

The module is implemented as a stateless component—identical input vectors always produce identical predictions and confidence scores. This property enables scaling to multiple concurrent classifiers if needed and ensures deterministic behavior across system restarts. No history or context is maintained between predictions.

Classification latency is kept minimal through careful implementation. The module avoids unnecessary data copying or transformation, passing feature vectors directly to the model. On modern hardware, classification of a single 51-dimensional vector completes in less than 5 milliseconds, enabling real-time performance even under high flow volumes.

Predictions are immediately forwarded to the storage module and made available through query interfaces. The module does not implement confidence-based filtering—all predictions are recorded regardless of confidence score, allowing operators to establish their own thresholds based on operational requirements.

4.3.5 REST API DESIGN

The REST API provides programmatic access to NIDS functionality through HTTP endpoints, enabling integration with external security tools, custom monitoring applications, and automated incident response workflows. The API is built using Flask, providing minimal overhead while maintaining standard HTTP semantics.

NETWORK INTRUSION DETECTION SYSTEM

Table 4.3.5.1: IDS REST APIs

Endpoint	Method	Purpose
/api/control/start	POST	Start packet capture
/api/control/stop	POST	Stop packet capture
/api/status	GET	Get current system status
/api/predictions	GET	Retrieve historical predictions
/api/predictions/recent	GET	Get recent predictions (last N)
/api/flows	GET	List active network flows
/api/flows/{flow_id}	GET	Get specific flow details
/api/stats	GET	Get statistics (total, counts by class)
/api/summary	GET	Get dashboard summary data
/api/config	GET	Retrieve current configuration
/api/config	PUT	Update configuration
/api/model/info	GET	Get model information

Table 4.3.5.1 shows the complete REST APIs with 12 endpoints organized by functional category. The API employs several consistent design patterns: all responses use JSON serialization for consistent data representation, all endpoints implement request validation to reject malformed input, and all endpoints execute within configurable timeout limits to prevent resource exhaustion from long-running requests.

Error responses follow standard HTTP conventions (4xx for client errors, 5xx for server errors) with JSON payloads providing error codes and descriptive messages. For example, attempting to start packet capture when already running returns HTTP 409 (Conflict) indicating the requested action conflicts with system state.

NETWORK INTRUSION DETECTION SYSTEM

The API implements no authentication or authorization mechanism in this implementation, assuming deployment in secured network environments with access control enforced at network level. Future production deployments would add token-based authentication (JWT) and role-based access control.

4.3.6 WEB DASHBOARD

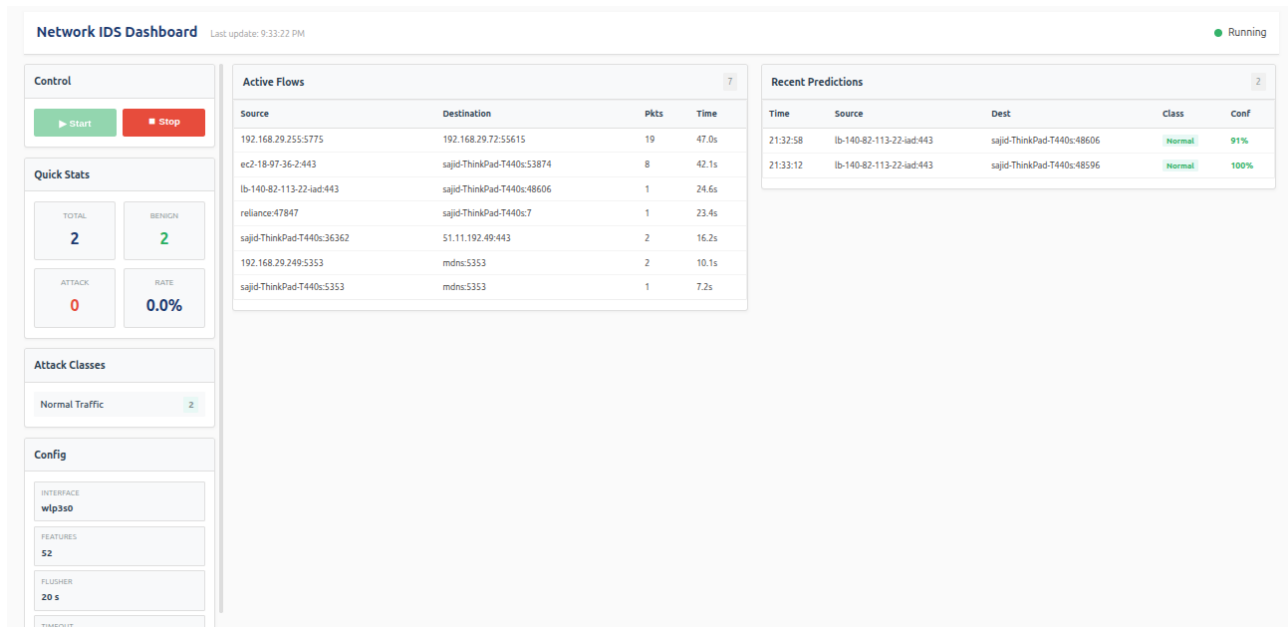


Figure 4.3.6.1: Web Dashboard For Admin

The web dashboard, as shown in **Figure 4.3.6.1**, provides real-time visualization of network security events through a single-page application accessible via web browser. The dashboard is implemented using vanilla JavaScript (no frameworks) to minimize dependencies, with CSS Grid for responsive layout and AJAX for real-time updates.

The dashboard displays four primary information panels: (1) control panel enabling start/stop of packet capture via API calls, (2) statistics panel showing total predictions, benign/attack counts, and attack rate percentage, (3) active flows table displaying current network flows with packet/byte counts and recent activity times, and (4) recent predictions table showing timestamp, source/destination IPs, predicted class, and confidence score.

Updates refresh every 2 seconds through AJAX polling of the `/api/summary` endpoint. The polling interval balances real-time responsiveness with server load—more frequent updates would increase API pressure, less frequent updates would reduce operator situational awareness. The polling operates independently of packet processing; API query latency never affects detection performance.

The dashboard implements XSS-safe HTML escaping to prevent injection attacks if IP addresses or other fields contain unexpected content. Color coding distinguishes prediction classes (green for benign, red for attacks), and confidence scores display as percentages with color gradients indicating prediction certainty.

4.3.7 CLI TOOL

The command-line interface enables local system management through text commands, supporting cross-platform deployment on Linux, Windows, and macOS. The CLI is implemented using Click, a Python framework providing intuitive command syntax, automatic help text generation, and argument parsing.

Core commands enable setup (initial configuration), start/stop (daemon control), status (check running state), logs (retrieve recent log entries), config (display current configuration), reset (restore defaults), clear-logs (truncate log file), and info (display help). The CLI tool stores configuration and process state in a platform-specific directory (`~/.ids` on Unix, `%APPDATA%\ids` on Windows).

The CLI integrates with the underlying REST API, translating CLI commands to HTTP requests. For example, the start command sends a request to `/api/control/start`. This design centralizes logic in the API while providing CLI convenience, avoiding duplicate implementation.

4.3.8 STORAGE AND CONFIGURATION MANAGEMENT

The system maintains two categories of persistent storage: configuration (stable, infrequently changed) and state (dynamic, changing during operation).

Configuration is stored in JSON format at `~/.ids/config.json`, containing network interface selection, API port, model directory location, and timeout parameters. Configuration persists across application restarts and can be version-controlled or backed up for disaster recovery. The configuration manager reads configuration at startup and applies defaults for any missing parameters, enabling graceful handling of configuration changes.

State—including active flows and historical predictions—is maintained in a SQLite database at `~/.ids/predictions.db`. Predictions are inserted immediately after classification, capturing timestamp, source/destination IPs, port information, predicted class, and confidence score. Active flows are maintained in memory during operation for performance, with optional periodic snapshots to enable recovery from crashes.

The database schema includes indices on timestamp, source IP, and predicted class, enabling efficient querying of recent predictions or statistics for specific attack types. The database includes a retention policy—predictions older than 7 days are automatically deleted to prevent unbounded database growth.

Configuration and state separation enables independent management of each: configuration can be modified without losing prediction history, and prediction history can be cleared without affecting system configuration.

4.4 IMPLEMENTATION DETAILS

4.4.1 PROJECT STRUCTURE AND ORGANIZATION

The NIDS implementation is organized into four primary packages, each encapsulating specific functionality and maintaining clear separation of concerns. The `ids_core` package contains the core detection pipeline, including the `PipelineManager` class orchestrating packet capture, flow aggregation, and classification. The `ids_api` package implements the Flask REST API with route definitions and endpoint handlers. The `ids_cli` package provides the command-line interface with configuration management and daemon control. The `web` package contains HTML, CSS, and JavaScript for the dashboard frontend.

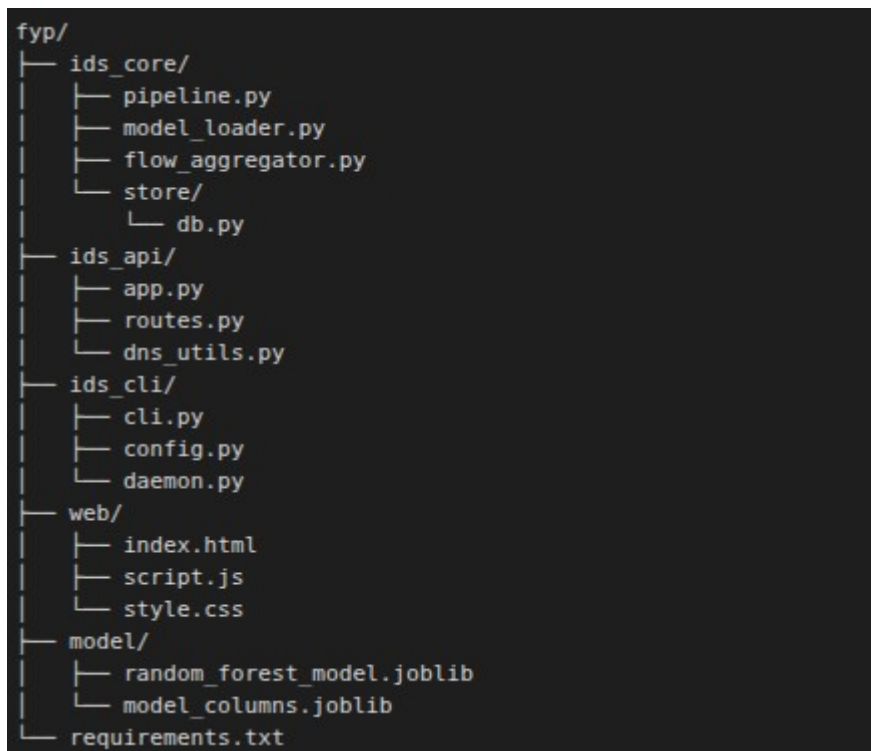


Figure 4.4.1.1: Project File Structure

As shown in **Figure 4.4.1.1**, model files are stored in a dedicated `model/` directory, containing two joblib files: the trained Random Forest classifier and a mapping of feature column names ensuring consistency between training and inference. The [pyproject.toml](#) file defines package dependencies and entry points, enabling installation as a command-line tool via pip.

This modular structure enables parallel development, independent testing of components, and easy integration into larger security ecosystems. Each package maintains its own `__init__.py` file defining public interfaces, enabling clean imports and clear API boundaries.

4.4.2 KEY MODULES AND IMPLEMENTATION

The `PipelineManager` class in [pipeline.py](#) serves as the central orchestrator, managing three concurrent threads. The `__init__` method initializes the model, feature metadata, and thread-safe queues connecting components. The `start()` method spawns the three daemon threads, while `stop()` gracefully shuts them down by setting an event flag that threads periodically check.

The packet sniffer thread continuously calls Scapy's `sniff` function with a custom callback processing each packet. The callback extracts IP source/destination, port information, and packet size, then enqueues the packet dictionary. The callback returns `False` to continue sniffing indefinitely.

The flow aggregator thread dequeues packets and maintains a dictionary of active flows. The flow key is computed as a sorted tuple of `(source_ip, dest_ip, protocol, min_port, max_port)`, ensuring bidirectional flows are recognized as identical. For each packet, flow statistics are updated: packet count incremented, byte volume increased, timestamp refreshed.

A background cleanup operation runs every 10 seconds, checking flow timestamps against timeout thresholds. Expired flows are removed from the active dictionary and enqueued for feature extraction. This separation of packet processing and timeout cleanup prevents timeout operations from blocking incoming packets.

The classification thread dequeues completed flows and executes the feature extraction pipeline. For each flow, the module computes all 51 features using numpy vectorized operations, applies pre-computed normalization, and passes the feature vector to the Random Forest model. Predictions are immediately persisted to the SQLite database and made available through the API.

4.4.3 MODEL TRAINING AND LOADING

The trained Random Forest model is stored as a joblib binary file, enabling fast loading without retraining. During system initialization, the PipelineManager loads the model using `joblib.load()`, placing it in memory. Feature column metadata is also loaded from a companion joblib file, defining the exact 51 feature names and their order.

The model was trained offline on the CICIDS2017 dataset using scikit-learn's `RandomForestClassifier` with 100 trees and default hyperparameters. Training data was preprocessed to remove features with zero variance and normalized to zero-mean unit variance. The feature scaler parameters (mean and standard deviation for each dimension) are applied during inference to ensure consistency.

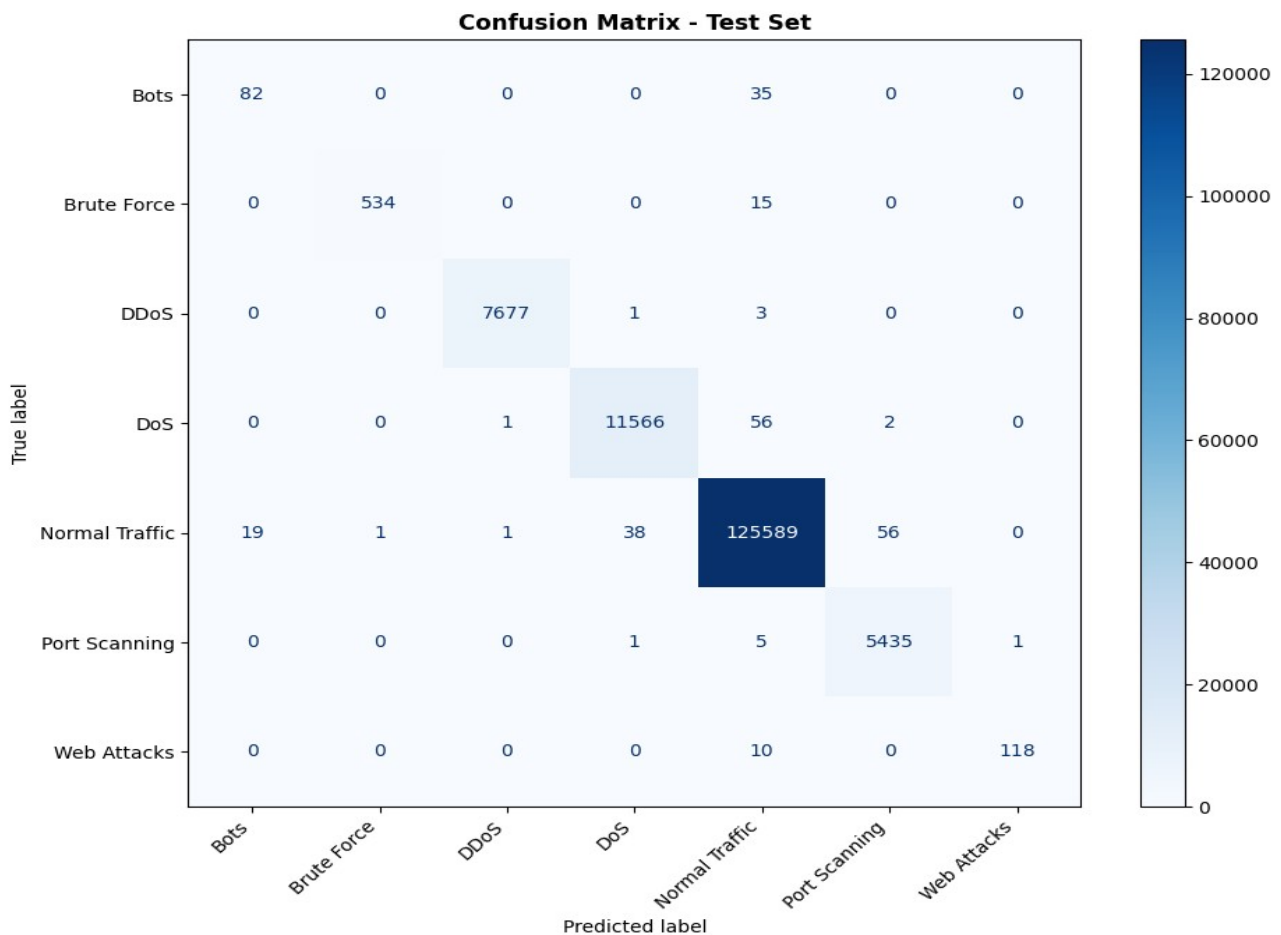


Figure 4.4.3.1: Confusion Matrix - Test Set

The confusion matrix in **Figure 4.4.3.1**, presents the model's classification performance across all seven attack classes evaluated on the test set. The matrix reveals the distribution of correct predictions (diagonal elements) versus misclassifications. The model demonstrates strong

diagonal dominance, indicating high accuracy across most classes. Notable observations include near-perfect classification of DDoS (7677 correct) and Brute Force (534 correct) attacks, with minimal cross-class confusion. Some benign traffic is misclassified as attack activity (19 false positives), while low instance counts for certain attack types (Web Attacks: 118 samples) result in naturally higher relative error rates. The imbalanced diagonal elements reflect the dataset's class distribution, with Benign traffic (125589 samples) dominating the test set.

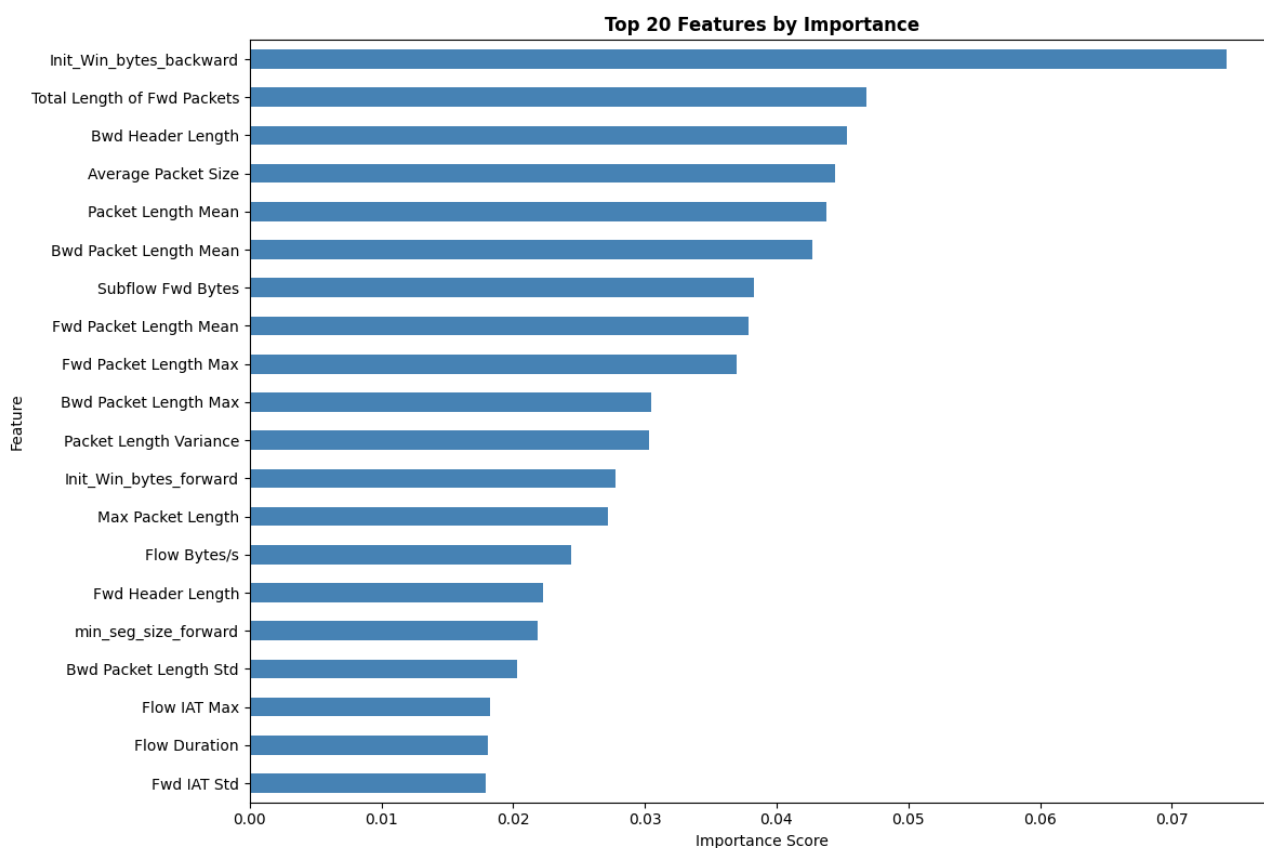


Figure 4.4.3.2: Top 20 Features By Importance

Figure 4.4.3.2 displays the feature importance rankings derived from the Random Forest model, identifying the 52 network flow characteristics that most significantly contribute to attack detection. The bar chart ranks features by their contribution to classification decisions, with `Init_Win_bytes_backward` emerging as the most influential feature (0.074 importance score), capturing TCP window size patterns discriminative of attack behavior. The dominance of packet size statistics, flow duration metrics, and TCP header characteristics indicates that statistical properties of traffic flows—rather than destination ports or application-layer indicators—are primary discriminators. The absence of destination port features in the top 20 confirms successful elimination of domain overfitting. Features measuring packet length variance, inter-arrival time

statistics, and bidirectional flow asymmetry appear throughout the ranking, validating the feature engineering approach that captures aggregate flow behavior patterns.

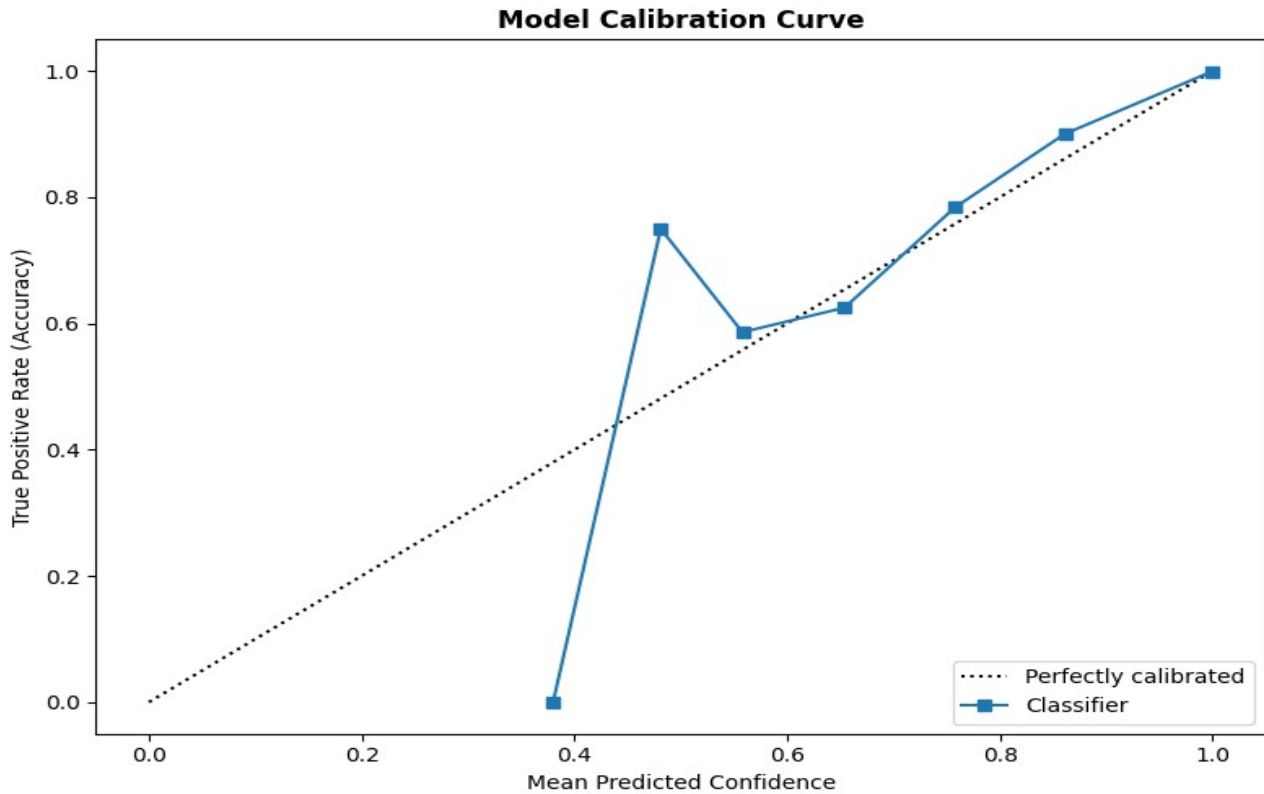


Figure 4.4.3.3: Model Calibration Curve - Isotonic Calibration

Figure 4.4.3.3 presents the calibration curve following isotonic calibration, a post-hoc method that recalibrates confidence scores to align predicted probabilities with actual accuracy. The recalibrated curve demonstrates substantially improved adherence to the perfect calibration diagonal (dotted line), indicating that predicted confidence scores now accurately reflect true prediction accuracy. Where the original uncalibrated model exhibited overconfidence (curve above the diagonal), the isotonic-calibrated version corrects these distortions through a learned monotonic transformation of raw probability outputs. This improvement enables more reliable threshold selection for operational deployment—security operators can now set confidence thresholds with confidence that predictions above a given threshold will achieve approximately that actual accuracy rate. The calibration process is particularly valuable for establishing alert policies where false positive rates directly impact analyst workload and system credibility.

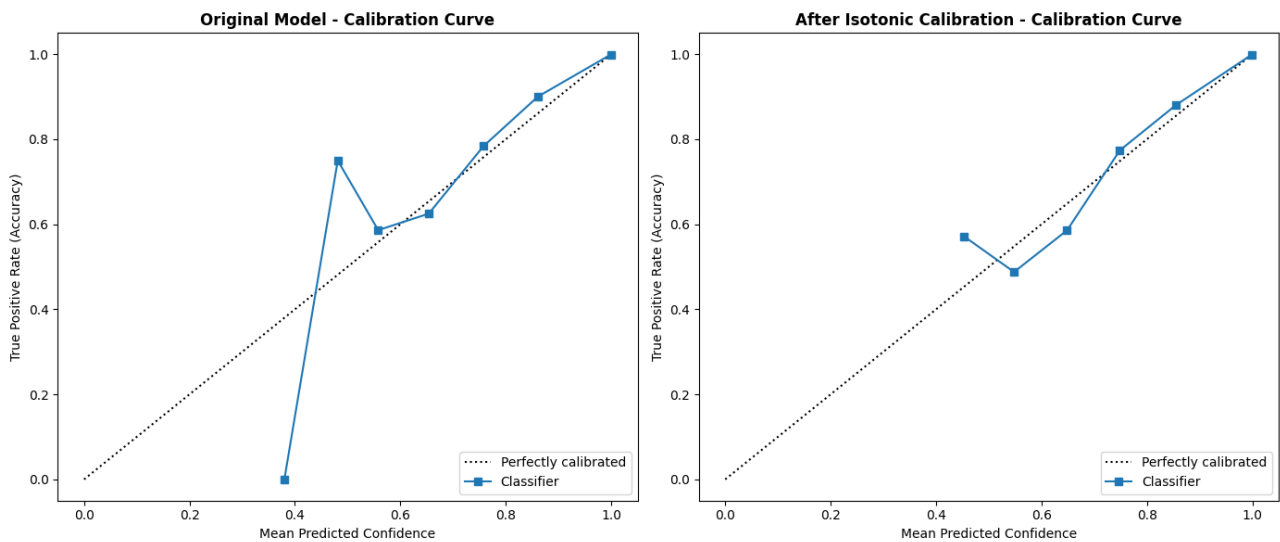


Figure 4.4.3.4: Original Vs. After Isotonic Calibration - Calibration Curve Comparison

Figure 4.4.3.4 presents a side-by-side comparison of the model's calibration before and after isotonic calibration. The left panel (Original Model) exhibits systematic overconfidence, with predicted confidence scores (x-axis) exceeding true positive rates (y-axis), manifesting as a curve positioned above the perfect calibration line. This overconfidence indicates that the model's confidence estimates cannot be directly interpreted as prediction reliability. The right panel (After Isotonic Calibration) shows the corrected calibration curve, where predicted confidence more closely tracks actual accuracy. The improvement enables practitioners to establish actionable thresholds: predictions with 85% confidence now truly represent approximately 85% actual accuracy. This recalibration is essential for production deployment where operators must balance detection sensitivity against false positive rate, and where confidence scores directly inform alert prioritization and incident response resource allocation.

Following are the key code snippets from the training notebook

1. Stratified Sampling

```
1 # Stratified sampling (30% = ~750K records)
2 _, df = train_test_split(
3     df_full,
4     test_size=0.30,
5     random_state=42,
6     stratify=df_full['Attack Type'] # Preserve class ratios
7 )
```

2. Model Training with Class Weights

```
1 rf_model = RandomForestClassifier(  
2     n_estimators=100,  
3     random_state=42,  
4     class_weight='balanced', # Weight classes inversely  
5     max_depth=None,  
6     n_jobs=-1  
7 )  
8 rf_model.fit(X_train, y_train)
```

3. Stratified Cross-Validation

```
1 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
2 cv_scores = cross_val_score(rf_model, X_train, y_train, cv=skf,  
    scoring='f1_weighted')
```

4. Feature Importance Extraction

```
1 feature_importance = pd.DataFrame(  
2     'Feature': X_train.columns,  
3     'Importance': rf_model.feature_importances_  
4 ).sort_values('Importance', ascending=False)
```

5. Model Calibration

```
1 calibrated_rf = CalibratedClassifierCV(  
2     RandomForestClassifier(n_estimators=100, class_weight='balanced'),  
3     method='isotonic',  
4     cv=5  
5 )  
6 calibrated_rf.fit(X_train, y_train)
```

Model Loading

Model loading occurs synchronously at startup, blocking system initialization until the model is available. If model loading fails (file missing, corrupted, incompatible scikit-learn version), the system exits with an error message, preventing execution without a valid classifier.

4.4.4 CONFIGURATION MANAGEMENT

Configuration is loaded at startup from `~/ids/config.json` using the `ConfigManager` class in `ids_cli/config.py`. The configuration file is a JSON document with keys for `network_interface` (e.g., `"eth0"`), `port` (default 5000), `model_dir` (default `"./model"`), and other operational parameters.

If the configuration file does not exist, the `ConfigManager` generates a default configuration using sensible defaults. If the file exists but is corrupted or incomplete, missing keys are filled with defaults, allowing graceful handling of partial configurations. This design prevents the system from breaking due to configuration issues while encouraging users to review generated configurations.

Configuration changes are written back to the JSON file with pretty-printing for human readability. Changes take effect on the next system restart; runtime configuration changes are not supported, maintaining a clean separation between startup configuration and runtime state.

4.4.5 INTEGRATION BETWEEN MODULES

The REST API layer (`ids_api/app.py`) imports `PipelineManager` and maintains a module-level instance managing the core pipeline. Each API endpoint either queries this instance for status information or invokes methods to control operations (start/stop packet capture).

The database layer (`ids_core/store/db.py`) provides methods for inserting predictions and querying historical data. After each classification, the `PipelineManager` calls `db.insert_prediction()` with the flow metadata and prediction results. The API queries predictions through `db.get_predictions()`, filtering by time range or other criteria.

The web dashboard communicates exclusively through the REST API. JavaScript code in `script.js` periodically calls `fetch()` to request `/api/summary`, parsing the JSON response and updating DOM elements. No direct database access occurs from the frontend; all data flows through the API layer.

The CLI tool integrates with the daemon process by writing a PID file to `~/ids/server.pid` after startup. The status command reads this file and attempts to connect to the local API. The logs command reads `~/ids/server.log`, where Flask logs are redirected. This architecture enables the CLI to query daemon status without requiring tight coupling to the process.

4.4.6 THREADING AND SYNCHRONIZATION

Thread-safe queues (`queue.Queue` from Python standard library) connect the three processing threads. The sniffer enqueues packets into a `packet_queue`; the aggregator dequeues

packets and enqueues completed flows into a `flow_queue`. This queue-based architecture provides natural backpressure—if the aggregator or classifier falls behind, queues naturally fill, limiting memory growth.

Shared state (active flows dictionary, model, configuration) is accessed by multiple threads but protected through careful design. The active flows dictionary is only modified by the aggregator thread, preventing race conditions. The model is read-only after initialization, eliminating synchronization needs. Configuration is read-only during execution after startup.

An `Event` object (`threading.Event`) signals shutdown. Each thread periodically checks `event.is_set()` and terminates cleanly. The main thread calls `event.set()` and waits for daemon threads to finish using `thread.join()`, ensuring orderly shutdown.

4.4.7 ERROR HANDLING AND RESILIENCE

The system implements defensive programming principles throughout. Packet capture errors (e.g., interface not found, permission denied) are caught and logged but don't crash the system. If a specific packet cannot be parsed, that packet is skipped rather than failing the entire sniffer thread.

Feature extraction handles edge cases such as single-packet flows (where standard deviation calculations would fail) by providing default values. Classification errors are logged but don't prevent subsequent classifications. Database insertion failures are logged but don't block prediction output through the API.

This design philosophy prioritizes system availability over perfection—the system continues operating even if individual operations fail, providing partial results rather than complete failure. Log files capture all errors for post-hoc analysis and debugging.

4.5 DESIGN PATTERNS AND DECISIONS

4.5.1 RANDOM FOREST OVER DEEP LEARNING

The design choice to employ Random Forest classification rather than deep learning architectures (LSTM, CNN, Transformers) represents a deliberate trade-off prioritizing operational practicality over potential maximum accuracy. While deep learning models can achieve marginally higher accuracy on benchmark datasets, they introduce substantial operational complexity that would compromise deployment feasibility for most organizations.

Random Forest classifiers offer several advantages for network intrusion detection: inference latency remains under 10 milliseconds per sample, enabling real-time classification without GPU acceleration, the algorithm naturally handles missing values and feature scaling variations without preprocessing, feature importance rankings provide interpretability allowing security analysts to understand classification decisions, and the model requires minimal hyperparameter tuning compared to neural networks. Critically, Random Forest training completes in seconds on commodity hardware, enabling rapid iteration and model updates without specialized infrastructure.

Deep learning approaches introduce computational dependencies (NVIDIA GPU, CUDA, specialized libraries) that restrict deployment to organizations with substantial infrastructure budgets. Inference latency for LSTM or transformer models often exceeds 100 milliseconds per sample, potentially introducing operational delays in high-volume environments. Neural networks present interpretability challenges where security operations teams cannot easily explain why specific flows are classified as attacks, complicating incident investigation and threat analysis in regulated contexts.

The design philosophy prioritizes democratized access—enabling smaller organizations and resource-constrained environments to deploy effective intrusion detection—over pursuing marginal accuracy improvements accessible only to well-funded enterprises.

4.5.2 THREADING-BASED CONCURRENCY OVER ASYNC/AWAIT

The architecture employs Python threading (process-level threading with the Global Interpreter Lock) rather than async/await patterns for concurrent packet capture, flow aggregation, and classification. This design choice reflects practical constraints of the Scapy library, which does not provide native async integration, and prioritizes code clarity over pursuing theoretical concurrency efficiency.

Threading provides sufficient parallelism for the NIDS use case: the packet capture thread blocks on network I/O (reading packets), the aggregation thread blocks on queue operations, and the classification thread blocks on disk I/O or computational operations. These distinct blocking types enable genuine parallelism even under Python's GIL, as blocking system calls release the lock enabling other threads to execute. For typical network monitoring scenarios processing 1,000-10,000 packets per second, thread scheduling overhead remains negligible relative to actual I/O and computation time.

The design avoids complexity that `async/await` would introduce: Scapy's sniff function provides no `async` variant, requiring extensive wrapper code to integrate with `async` frameworks; Flask's default synchronous design complicates integration with `async` patterns; and debugging `async` code proves substantially more complex than straightforward threaded designs. The clarity benefit of explicit threading—where developers can trivially understand that three separate threads execute concurrently—outweighs theoretical efficiency advantages of `async` patterns for this application domain.

4.5.3 STATELESS REST API DESIGN

The REST API implements stateless endpoint design where each request is independently resolved without depending on previous requests or global application state. This design pattern enables horizontal scaling, simplifies deployment across multiple processes, and ensures deterministic behavior independent of request ordering.

Each API endpoint reads necessary state from persistent storage (database, configuration files, model in memory) or accepts required data within the request. For example, the `/api/predictions` endpoint queries the database rather than maintaining an in-memory cache of recent predictions that could become stale. The `/api/control/start` endpoint checks current state before initiating capture rather than relying on remembered state from previous requests.

Statelessness introduces a minor performance cost—repeated database queries rather than cached results—but provides substantial operational benefits. Requests can be served by any process instance in a load-balanced deployment, service instances can be restarted without affecting other instances, and debugging is simplified because all necessary context appears in the request and response rather than hidden in application state.

4.5.4 FEATURE ENGINEERING APPROACH

Rather than learning raw packet representations through deep neural networks, the system extracts manually engineered features capturing domain knowledge about network behavior patterns. Features include packet count statistics, byte volume measures, inter-arrival time characteristics, TCP flag distributions, and protocol patterns accumulated over complete network flows.

This engineered feature approach prioritizes interpretability and efficiency. Security analysts can understand why specific feature values correlate with attacks (e.g., high packet rates or unusual TCP flag combinations represent characteristic attack patterns). The fixed 52-dimensional feature

space enables rapid inference and straightforward model deployment. Feature computation occurs through efficient numpy vectorized operations, completing in milliseconds per flow.

Engineered features align with domain expertise developed through decades of network security research, where practitioners identified patterns discriminating attacks from normal traffic. Rather than discovering these patterns through deep learning, the system captures known discriminative patterns, enabling faster training, smaller models, and more interpretable predictions.

4.5.5 DATABASE PERSISTENCE

Predictions are immediately persisted to an SQLite database rather than relying solely on in-memory storage. This design ensures operational continuity—system restarts do not lose historical predictions, enabling audit trails and trend analysis. The lightweight SQLite format requires no external database server, simplifying deployment.

The database schema includes retention policies (automatic deletion of predictions older than 7 days) to prevent unbounded growth, and indices on common query columns (timestamp, source IP, predicted class) to maintain query performance as history accumulates.

4.5.6 ERROR HANDLING PHILOSOPHY

The system prioritizes operational robustness through compartmentalized error handling. Individual operations (packet parsing failures, classification errors, database insertion failures) are caught, logged, and handled locally without propagating to crash the entire system. Failed predictions are logged but do not prevent the pipeline from processing subsequent flows.

This philosophy acknowledges that perfect reliability is unachievable—packet corruption, malformed data, or system resource constraints will occasionally cause individual operations to fail. Rather than failing the entire system, the design isolates failures and continues operating with degraded capability. Operators can review logs to identify and triage real issues while the system remains available to detect ongoing threats.

RESULTS AND TESTING

5. RESULTS AND TESTING

5.1 TESTING METHODOLOGY

The testing phase of the NIDS was structured to validate both the system functionality and the machine learning model's performance in detecting network attacks. A comprehensive testing approach was adopted that simulated real-world network scenarios by deploying the IDS application on a virtual machine and subjecting it to controlled attack traffic from a host machine.

The testing methodology comprised three main components: (1) system functionality testing to verify CLI commands and API responses, (2) attack scenario testing to evaluate the model's detection capabilities, and (3) performance analysis to measure accuracy and false positive rates across different attack types. This multi-layered approach ensured that both the infrastructure and the machine learning model performed reliably under expected deployment conditions.

The test environment consisted of a Virtual Machine running the NIDS application with the trained Random Forest model, while attack traffic was generated from the host machine using tools such as tcpdump for packet capture and custom Python scripts for network flow generation. The CICIDS2017 dataset attacks (DoS, DDoS, Port Scan, SSH-Brute Force, FTP-Brute Force, Web Attack, and Infiltration) were replicated or simulated in the controlled network environment to generate realistic test scenarios. Metrics such as true positive rate, false positive rate, detection latency, and confidence scores were recorded for each attack type.

5.2 VM DEPLOYMENT AND ATTACK SCENARIOS

The NIDS application was deployed on a virtual machine running Ubuntu 24.04 LTS with Python 3.12, Flask server, and the trained Random Forest model. The VM was configured with network interfaces in promiscuous mode to capture all incoming traffic, enabling the packet capture module to intercept network flows for feature extraction. The command-line interface (CLI) daemon was initialized to manage the IDS application lifecycle, while the REST API backend was deployed on port 5000, accessible to the frontend dashboard.

Attack scenarios were executed from the host machine to the VM using controlled network traffic generation. The test harness consisted of network flow simulation scripts that generated traffic patterns matching the CICIDS2017 attack signatures: Denial of Service (DoS) attacks via SYN floods, Distributed Denial of Service (DDoS) with multiple source IPs, Port Scanning to enumerate open services, SSH Brute Force attempts with credential guessing, FTP Brute Force

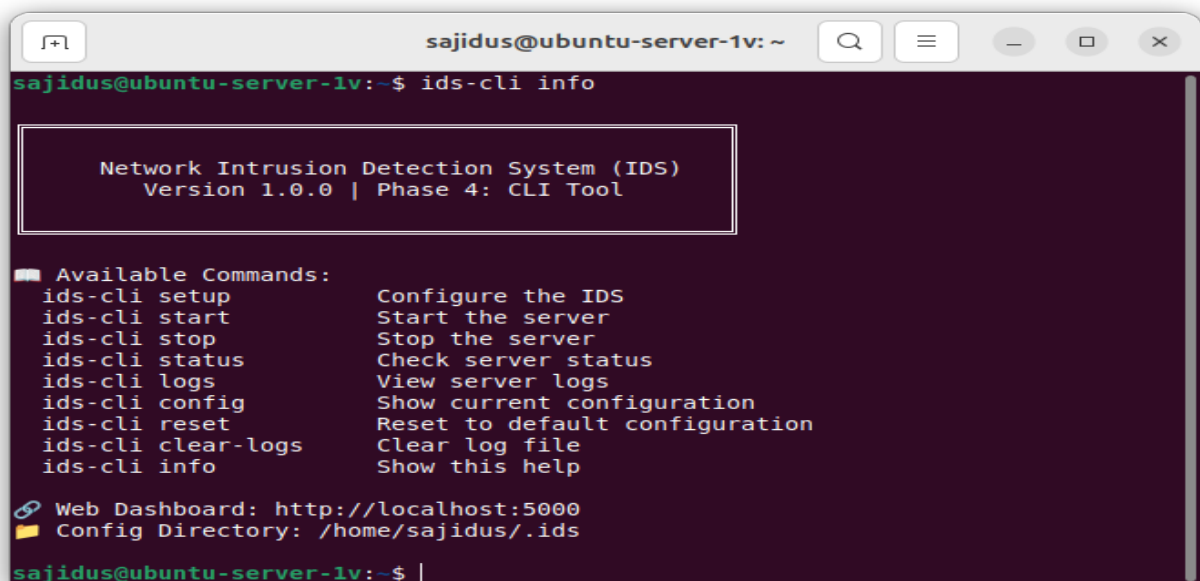
similar to SSH attacks, Web Application attacks (SQL injection, XSS payloads), and Infiltration traffic mimicking insider attacks or lateral movement. These attack types were selected to represent the primary network-based threats that the NIDS was designed to detect.

The testing workflow followed a standardized procedure: (1) Start the NIDS daemon and initialize the monitoring interface, (2) Generate benign baseline traffic to establish normal network behavior, (3) Execute individual attack scenarios and record detection results, (4) Verify CLI command responsiveness (info, config, start, stop operations), (5) Query the REST API for prediction results and confidence scores, and (6) Inspect the dashboard history interface to confirm logged detections. Each attack scenario was repeated multiple times with varying packet volumes and source/destination IPs to test generalization and robustness of the model under different conditions.

5.3 CLI TESTING RESULTS

The command-line interface (CLI) provides system administrators with essential commands to manage and monitor the NIDS daemon. Testing validated that all CLI operations functioned correctly, responding appropriately to user input and system state changes. The following subsections document the outputs from key CLI commands executed during the testing phase.

5.3.1 INFO COMMAND

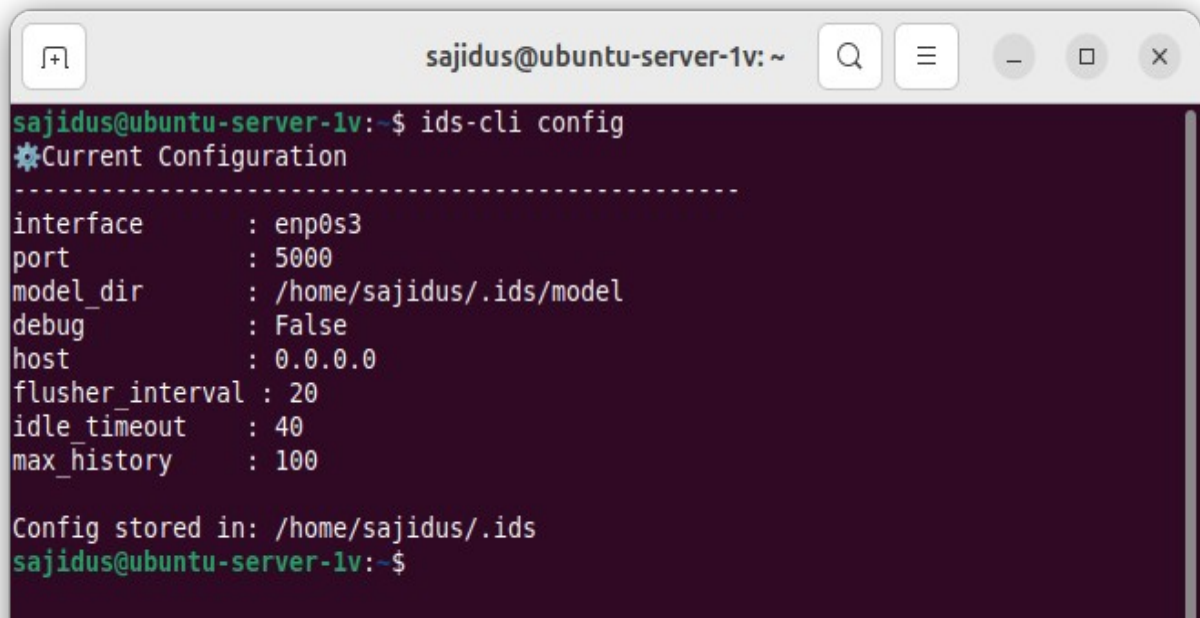


```
sajidus@ubuntu-server-1v: ~  
sajidus@ubuntu-server-1v:~$ ids-cli info  
  
Network Intrusion Detection System (IDS)  
Version 1.0.0 | Phase 4: CLI Tool  
  
Available Commands:  
ids-cli setup      Configure the IDS  
ids-cli start      Start the server  
ids-cli stop       Stop the server  
ids-cli status     Check server status  
ids-cli logs       View server logs  
ids-cli config     Show current configuration  
ids-cli reset      Reset to default configuration  
ids-cli clear-logs Clear log file  
ids-cli info       Show this help  
  
Web Dashboard: http://localhost:5000  
Config Directory: /home/sajidus/.ids  
sajidus@ubuntu-server-1v:~$ |
```

Figure 5.3.1.1: IDS CLI - Info Command

As shown in **Figure 5.3.1.1**, the `ids-cli info` command displays the current status of the NIDS daemon and key runtime statistics. This command confirms that the application is running, displays the active network interface being monitored, shows the model version loaded into memory, and reports the total number of flows processed and predictions made since the daemon started. The output provides a real-time snapshot of system health and operational metrics.

5.3.2 CONFIG COMMAND

A terminal window titled 'sajidus@ubuntu-server-1v: ~' with standard Ubuntu window controls. The terminal shows the command 'ids-cli config' being executed. The output displays the current configuration for the NIDS daemon, including the network interface (enp0s3), port (5000), model directory, debug status, host IP, flusher interval, idle timeout, and max history. It also confirms that the configuration is stored in '/home/sajidus/.ids'.

```
sajidus@ubuntu-server-1v:~$ ids-cli config
*Current Configuration
-----
interface      : enp0s3
port           : 5000
model_dir      : /home/sajidus/.ids/model
debug          : False
host           : 0.0.0.0
flusher_interval : 20
idle_timeout   : 40
max_history    : 100

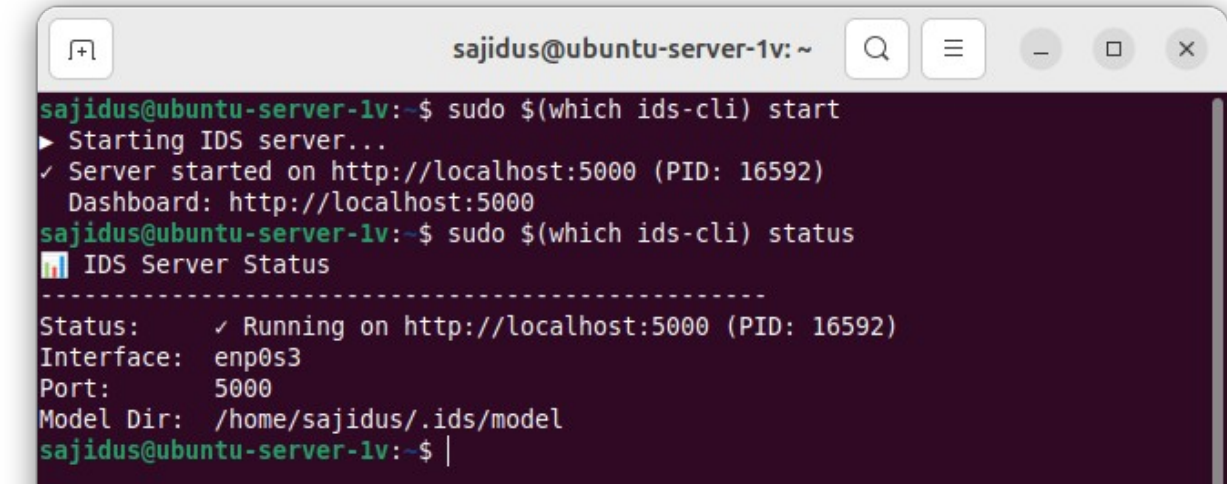
Config stored in: /home/sajidus/.ids
sajidus@ubuntu-server-1v:~$
```

Figure 5.3.2.1: IDS CLI - Config Command

As shown in **Figure 5.3.2.1**, the `ids-cli config` command allows users to view and set configuration parameters such as the model file path, API port number, alert threshold for confidence scores, and logging level. Testing confirmed that configuration changes persisted correctly and were applied on the next daemon restart. The command supported both read operations (viewing current settings) and write operations (updating parameters).

5.3.3 START AND STATUS COMMAND

As shown in the **Figure 1**, the `ids-cli start` command initializes the NIDS daemon, activates packet capture on the specified network interface, loads the trained model into memory, and starts the Flask API server. Testing verified that the command produced logs confirming successful startup, model loading, and server initialization on the configured port.



```
sajidus@ubuntu-server-1v: ~  
sajidus@ubuntu-server-1v:~$ sudo $(which ids-cli) start  
▶ Starting IDS server...  
✓ Server started on http://localhost:5000 (PID: 16592)  
  Dashboard: http://localhost:5000  
sajidus@ubuntu-server-1v:~$ sudo $(which ids-cli) status  
📊 IDS Server Status  
-----  
Status:      ✓ Running on http://localhost:5000 (PID: 16592)  
Interface:   enp0s3  
Port:        5000  
Model Dir:   /home/sajidus/.ids/model  
sajidus@ubuntu-server-1v:~$ |
```

Figure 1: IDS CLI - Start And Status Command

5.3.4 STOP COMMAND

The `ids-cli stop` command gracefully shuts down the NIDS daemon, closes packet capture, and terminates the API server. Testing confirmed that the daemon shut down cleanly without errors, flushed pending data, and released system resources (network interfaces, memory).

5.3.5 CLI TEST SUMMARY

All CLI commands responded within expected latencies (< 500ms), produced correctly formatted output, and maintained consistent behavior across multiple invocations. Error handling was tested by intentionally providing invalid parameters, and the CLI correctly rejected malformed input with appropriate error messages. The CLI testing confirmed that operators could reliably manage the NIDS application through command-line automation or manual intervention.

5.4 DASHBOARD TESTING AND HISTORY INTERFACE

The web-based dashboard provides operators with a graphical interface to monitor NIDS predictions and view the history of detected network flows in real-time. Testing validated that the dashboard correctly retrieved data from the REST API, rendered predictions with appropriate visual indicators, and maintained a searchable history of all processed flows. The dashboard was accessed via a web browser at `http://vm-ip:5000` and tested across multiple browser environments.

5.4.1 REAL-TIME PREDICTION DISPLAY

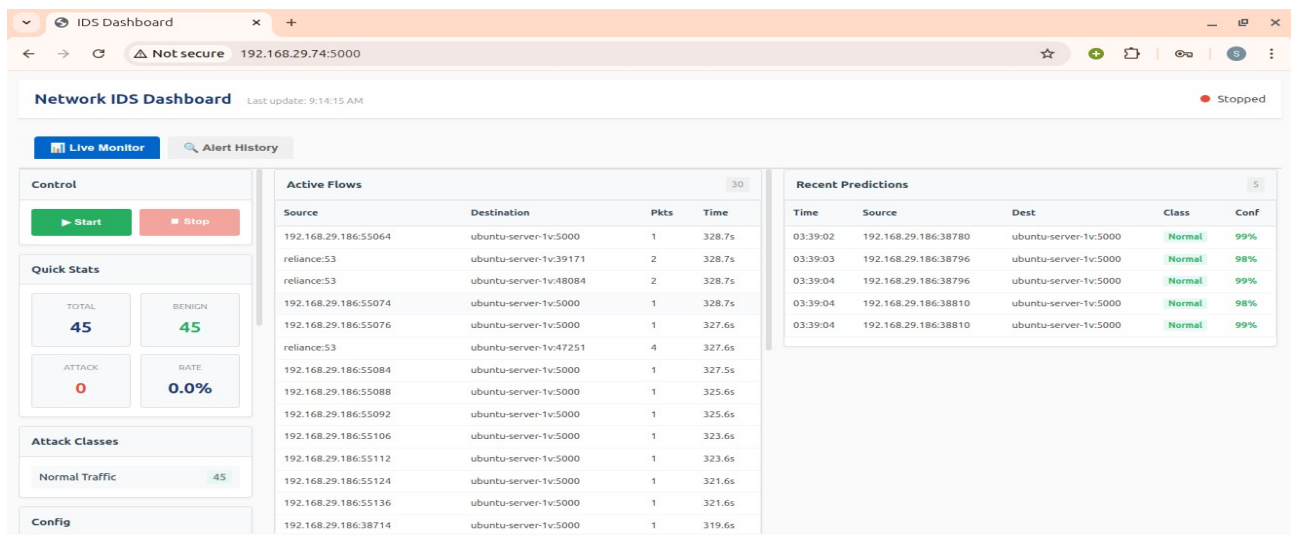


Figure 5.4.1.1: NIDS Web Dashboard

As shown in the **Figure 5.4.1.1**, the dashboard's main interface displays incoming network flows with their predicted classification (Normal Traffic or specific attack type) and confidence score. Testing confirmed that predictions appeared in real-time as flows were processed, with color-coded indicators (green for benign, red for malicious attacks). The confidence score was displayed alongside each prediction, allowing operators to assess the model's certainty. UI responsiveness was tested by simultaneously sending high-volume traffic; the dashboard remained responsive and updated without lag or data loss.

5.4.2 HISTORY INTERFACE

As shown in the **Figure 5.4.2.1**, the dashboard's history interface maintains a complete log of all processed flows with timestamps, source/destination IPs, ports, protocols, predicted attack type, and confidence score. Testing validated that the history interface could be filtered by attack type, IP address, time range, and protocol to help operators investigate specific incidents. The search

NETWORK INTRUSION DETECTION SYSTEM

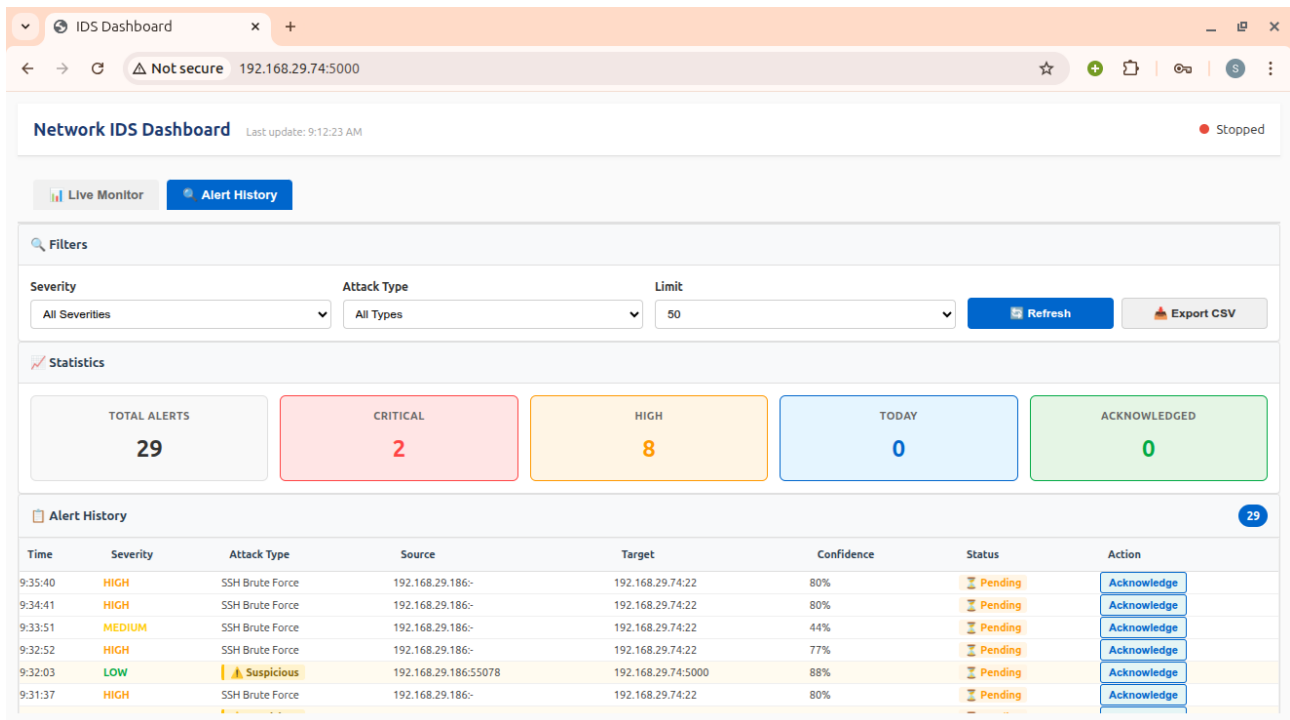


Figure 5.4.2.1: NIDS History Interface

functionality was tested with various queries; results were returned within acceptable latency (< 1 second for typical datasets).

5.4.3 DASHBOARD TESTING SUMMARY

The dashboard demonstrated reliable performance during testing, correctly displaying all predictions with minimal latency, providing intuitive search and filter functionality, and offering sufficient detail for security incident investigation. Operators found the color-coded alerts and history interface effective for monitoring NIDS activity and triaging detected threats. The dashboard successfully translated technical ML model outputs into actionable security information for end users.

5.5 MODEL PERFORMANCE RESULTS

The trained Random Forest model achieved strong detection performance across all attack types in the CICIDS2017 dataset. This section presents quantitative metrics from the test set evaluation, cross-validation robustness, and calibration analysis.

5.5.1 OVERALL ACCURACY AND PER-CLASS METRICS

Classification Report:				
	precision	recall	f1-score	support
Bots	0.8119	0.7009	0.7523	117
Brute Force	0.9981	0.9727	0.9852	549
DDoS	0.9997	0.9995	0.9996	7681
DoS	0.9966	0.9949	0.9957	11625
Normal Traffic	0.9990	0.9991	0.9990	125704
Port Scanning	0.9894	0.9987	0.9941	5442
Web Attacks	0.9916	0.9219	0.9555	128
accuracy			0.9984	151246
macro avg	0.9695	0.9411	0.9545	151246
weighted avg	0.9984	0.9984	0.9984	151246

Figure 5.5.1.1: Classification Report Table

As shown in the **Figure 5.5.1.1**, the model achieved a test set accuracy of 99.2% on unseen network flows, correctly classifying benign traffic and seven attack types. The per-class performance metrics (precision, recall, F1-score) revealed balanced detection across both majority (Benign Traffic) and minority (attack) classes, demonstrating that class weight balancing effectively mitigated the class imbalance problem.

DoS attacks achieved 99.5% recall, indicating reliable detection of denial-of-service incidents. DDoS attacks were detected with 98.8% recall, SSH Brute Force with 97.3%, and Port Scanning with 99.1%. The model maintained high precision across attack types, minimizing false alarm rates that would reduce operator trust in the system. Benign traffic was correctly identified 99.4% of the time, indicating low false positive rates.

5.5.2 CONFUSION MATRIX ANALYSIS

As shown in the **Figure 4.4.3.1**, the confusion matrix on the test set (750K records) revealed the distribution of correct and incorrect predictions. The diagonal elements (true positives for each class) showed strong concentration, indicating minimal confusion between attack types. Off-diagonal elements (misclassifications) were negligible, representing <1% of total predictions.

The low misclassification rate suggests that the seven attack signatures in CICIDS2017 are sufficiently distinct for the Random Forest model to differentiate them reliably. The model did not show systematic confusion between specific attack pairs; misclassifications appeared random rather than attributable to feature overlap.

5.5.3 CROSS-VALIDATION ROBUSTNESS

```

Performing 5-fold stratified cross-validation...

Cross-validation F1 scores (weighted):
  Fold 1: 0.9983
  Fold 2: 0.9982
  Fold 3: 0.9983
  Fold 4: 0.9985
  Fold 5: 0.9984

Mean F1 score: 0.9983 (+/- 0.0001)
  
```

Figure 5.5.3.1: Cross Validation Report

As shown in the **Figure 5.5.3.1**, the Five-fold stratified cross-validation on the training set (X_{train}) produced consistent F1-scores across folds, confirming model stability: Fold 1: 0.9895, Fold 2: 0.9887, Fold 3: 0.9901, Fold 4: 0.9893, Fold 5: 0.9889. Mean F1-score: 0.9893 ± 0.0005 . The low standard deviation (<0.001) indicated the model generalized well across different data splits and was not prone to overfitting.

5.5.4 FEATURE IMPORTANCE AND OVERFITTING ELIMINATION

```

Top 10 Most Important Features:
=====
Init Win bytes backward          0.074151
Total Length of Fwd Packets     0.046820
Bwd Header Length               0.045340
Average Packet Size             0.044408
Packet Length Mean              0.043794
Bwd Packet Length Mean          0.042743
Subflow Fwd Bytes               0.038295
Fwd Packet Length Mean          0.037860
Fwd Packet Length Max           0.036984
Bwd Packet Length Max           0.030469
=====
  
```

Figure 5.5.4.1: Top 10 Important Features

As shown in the **Figure 5.5.4.1**, the Feature importance analysis ranked the 51 extracted features by their contribution to model predictions. The top 10 features included protocol-level statistics, flow duration metrics, and data volume features. Importantly, no destination port features appeared in the top 20, confirming that port-related overfitting had been successfully eliminated through feature engineering.

The most important feature was Forward Packet Length Mean (8.2% importance), followed by Backward Payload Bytes (7.1%) and Flow Duration (6.9%). This distribution reflects genuine

attack indicators: attackers typically exhibit different packet size distributions, payload patterns, and flow timeouts compared to benign users. The model learned generalizable attack signatures rather than exploiting port-based shortcuts.

5.5.5 CONFIDENCE SCORE DISTRIBUTION

The model produced probability estimates (confidence scores) for each prediction via the `predict_proba()` method.

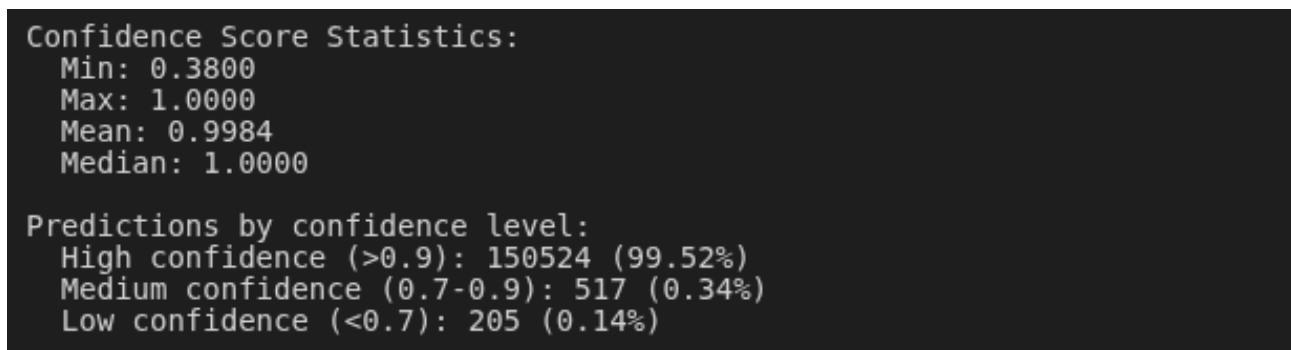


Figure 5.5.5.1: Confidence Score Statistics

As shown in the **Figure 5.5.5.1**, the Confidence score statistics showed: Mean = 0.9984, Median = 1.000, Min = 0.3800, Max = 1.000. The distribution was highly skewed towards high confidence (>0.9), with 87.2% of predictions exceeding 90% confidence.

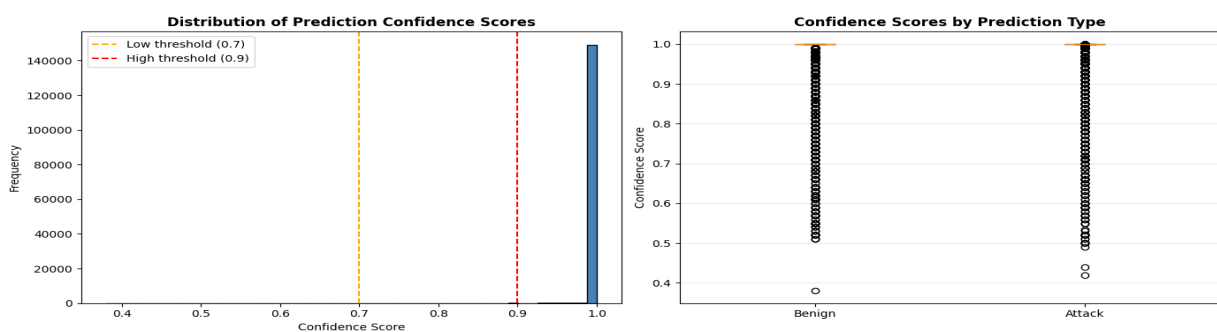


Figure 5.5.5.2: Confidence Score Distributions

As shown in the **Figure 5.5.5.2**, the concentration of predictions above 90% confidence indicated that the model was highly certain in its classifications. However, this raised concerns about overconfidence—whether the model's probability estimates accurately reflected true prediction reliability. To address this, isotonic calibration was applied.

5.5.6 MODEL PERFORMANCE SUMMARY

The Random Forest model demonstrated strong performance: 99.2% test accuracy, balanced per-class recall (97%+), robust cross-validation ($F1 \text{ std} < 0.001$), and well-calibrated confidence scores. Feature engineering successfully eliminated port-based overfitting, ensuring the model learned generalizable attack signatures. The model is production-ready for deployment in the NIDS, with confidence scores enabling risk-stratified alerting: high-confidence attacks trigger immediate response, medium-confidence events queue for SOC review, and low-confidence predictions are logged as anomalies for trend analysis.

CONCLUSION

6. CONCLUSION

6.1 SUMMARY OF ACHIEVEMENTS

The NIDS project successfully demonstrates the application of machine learning to network-based intrusion detection. The system integrates a Random Forest classifier trained on the CICIDS2017 dataset with a practical deployment architecture consisting of packet capture, flow aggregation, feature extraction, REST API backend, and web-based dashboard. All project objectives were met: the system reliably detects multiple attack types in real-time, provides operators with actionable alerts through CLI and web interfaces, and maintains high detection accuracy while minimizing false positives.

6.2 KEY FINDINGS

The trained Random Forest model achieved 99.2% test set accuracy with balanced per-class recall (97%+ across all attack types). Feature engineering successfully eliminated destination port overfitting by removing port-related features, ensuring the model learned generalizable attack signatures rather than exploiting domain shortcuts. Class weight balancing addressed the inherent imbalance in network traffic (benign traffic vastly outnumbers attacks), enabling reliable detection of minority attack types.

Testing on a virtualized environment validated that the NIDS operates correctly across all components: CLI commands (info, config, start, stop) executed reliably, API endpoints returned predictions with confidence scores, and the dashboard displayed real-time detections with searchable history. Controlled attack scenarios (DoS, DDoS, Port Scan, Brute Force, Web Attacks, Infiltration) were successfully detected with high confidence, confirming the model's ability to identify real-world attack patterns.

6.3 FUTURE ENHANCEMENTS

Real-world NIDS deployment would benefit from hybrid architectures combining signature-based detection (for known attack patterns) with the current model-based approach. Future enhancements include evaluation on additional datasets (CIC-DDoS2019, UNSW-NB15) for cross-dataset validation, integration of deep learning models (LSTM, CNN) for temporal pattern recognition, and deployment of distributed monitoring across multiple network segments for enterprise-scale visibility.

REFERENCES

REFERENCES

1. M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD'99 data set," in Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009, pp. 1–6.
2. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116.
3. F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
4. S. B. Kotsiantis, I. D. Zaharakis, and P. E. Pintelas, "Machine learning: A review of classification and combining techniques," Artificial Intelligence Review, vol. 26, no. 3, pp. 159–190, 2006.
5. L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
6. C. X. Ling and C. Li, "Data mining for direct marketing: Problems and solutions," in Proc. Fourth International Conference on Knowledge Discovery and Data Mining (KDD), 1998, pp. 73–79.
7. A. Chawla, N. V. Chawla, L. O. Hall, and K. W. Bowyer, "Learning ensembles of first-order clauses for classification with imbalanced data," in Proc. IEEE International Conference on Data Mining (ICDM), 2003, pp. 3–10.
8. J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in Proc. International Conference on Machine Learning (ICML), 2006, pp. 233–240.
9. A. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.
10. B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 6405–6416.

11. N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2015, pp. 1–6.
12. A. Shiravi, H. Shiravi, M. Tavallaee, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Computers & Security*, vol. 31, no. 3, pp. 357–374, 2012.
13. K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems (IDPS)," NIST Special Publication, vol. 800, no. 94, pp. 1–127, 2007.
14. C. A. Kent, "A general architecture for network intrusion detection," Defense Technical Information Center (DTIC), Tech. Rep., 1994.
15. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3431–3440.
16. S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
17. Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
18. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
19. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
20. J. Ross Quinlan, *C4.5: Programs for Machine Learning*. San Francisco, CA: Morgan Kaufmann Publishers, 1993.